CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

KIẾN TRÚC MICROSERVICE

Lợi ích và thách thức của kiến trúc microservice.
Thiết kế ứng dụng thương mại điện tử với kiến trúc microservice.

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

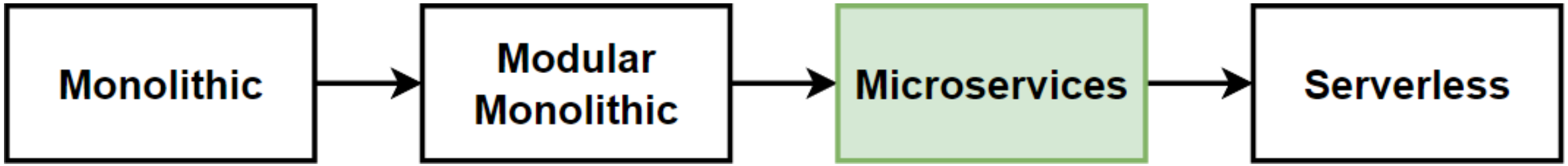
Microservices Architecture

Benefits and Challenges of Microservices Architecture

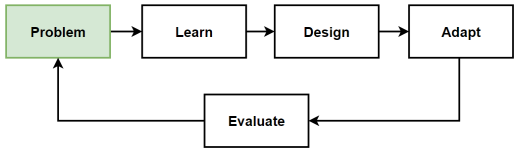
When to use Microservices Architecture

Design our E-Commerce application with Microservices Architecture

Architecture Design Journey



Problem: Scale and Deploy Independently

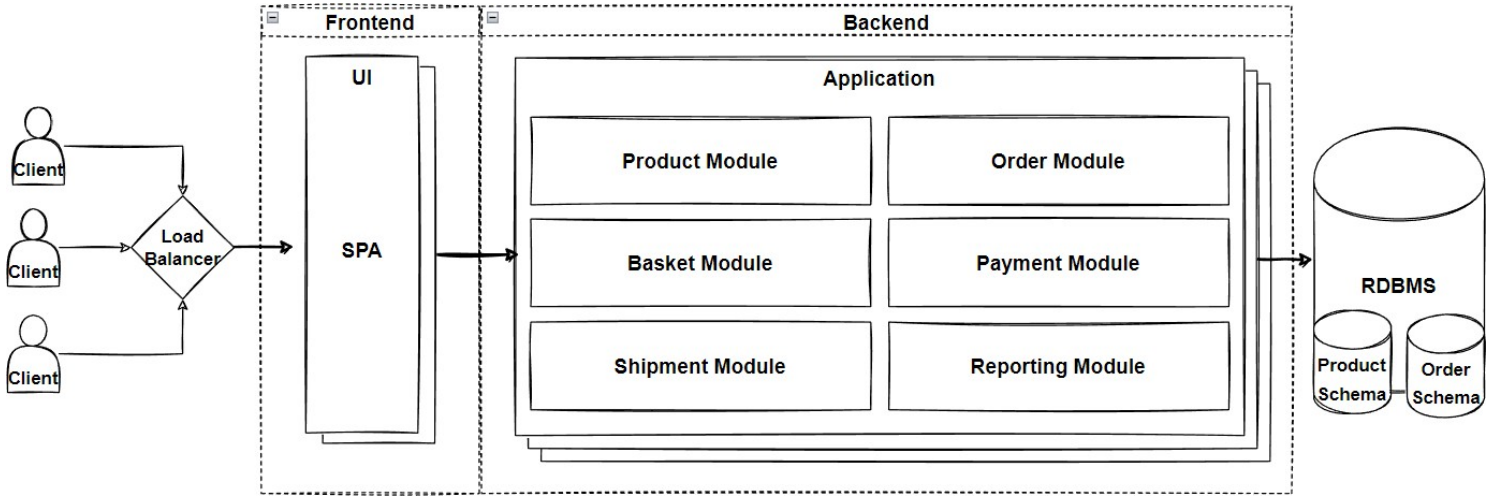


Problems

- Our E-Commerce Business is growing
- Business teams are **separated teams** as per departments; Product, Sale, Payment.
- Teams wants to agile and **add new features immediately** to compete the market
- Innovate and experiment with new features as soon as possible
- **Deploy features immediately**, not waiting for deployment dates
- **Flexible scale** for **market peek times** like blackfriday sales
- Handle and process **millions of request** in a acceptable latency with better performance.
- Required not only technology change but also **organizational change is mandatory**.

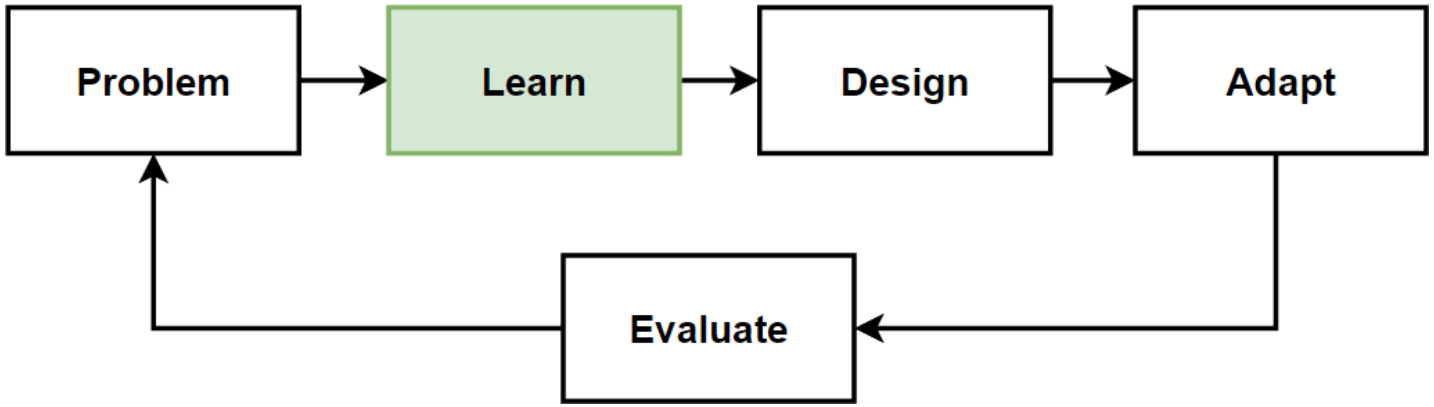
Solutions

- Microservices Architecture

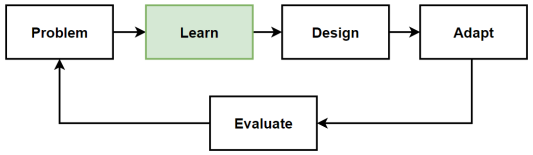


Learn: Microservices Architecture

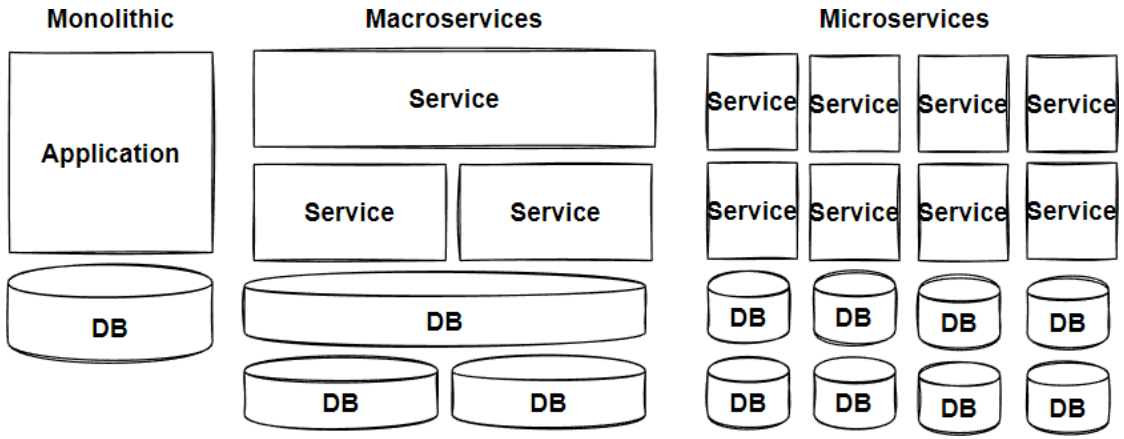
- Microservices Architecture
- When to use Microservices Architecture
- Benefits of Microservices Architecture
- Challenges of Microservices Architecture
- Microservices Architecture Pros-Cons
- Reference Architectures of Microservices



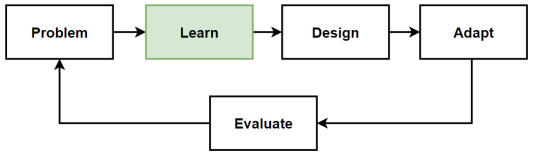
What are Microservices ?



- **Microservices** are **small**, **independent**, and **loosely coupled** services that can work together.
- Each service is a **separate codebase**, which can be managed by a small development team.
- Microservices **communicate** with each other by using **well-defined APIs.**
- Microservices can be **deployed independently** and **autonomously**.
- Microservices can work with many different technology stacks which is **technology agnostic**.
- Microservices has its **own database** that is not shared with other services.



What is Microservices Architecture ?

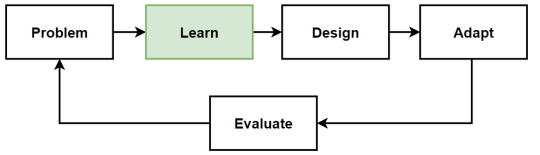


- From Martin Fowlers Microservices article;

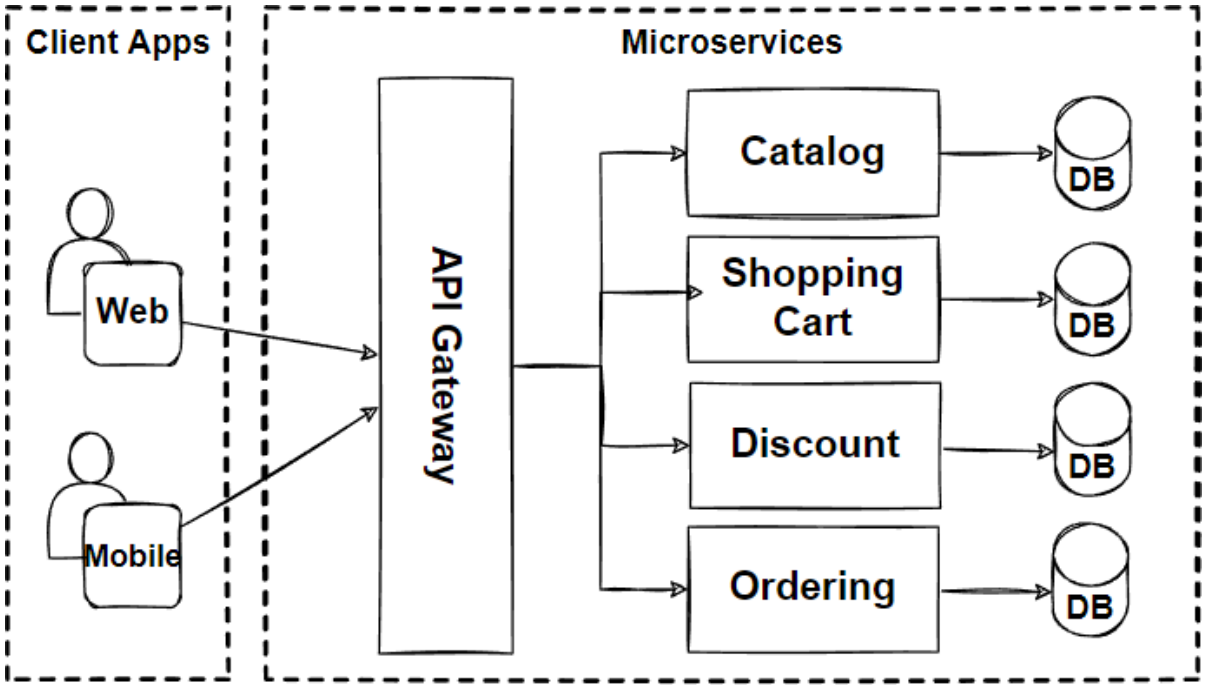
*The microservice architectural style is an approach to developing a single application as a **suite of small services**, each running in **its own process** and **communicating** with lightweight mechanisms, often an **HTTP** or **gRPC API**.*

- Microservices are **built around business capabilities** and **independently deployable** by **fully automated** deployment process.
- Microservices architecture **decomposes** an application into **small independent services** that communicate over **well-defined APIs**. Services are owned by **small**, **self-contained teams**.
- Microservices architecture is a **cloud native architectural approach** in which services composed of many **loosely coupled** and **independently deployable** smaller components.
- Microservices have their **own technology stack**, communicate to each other over a combination of **REST APIs**, are organized by **business capability**, with the **bounded contexts**.
- Following **Single Responsibility Principle** that referring separating responsibilities as per services.

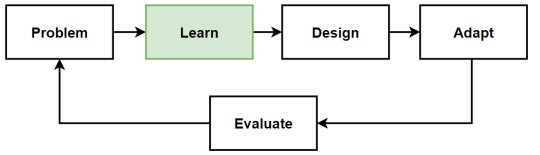
Microservices Characteristics



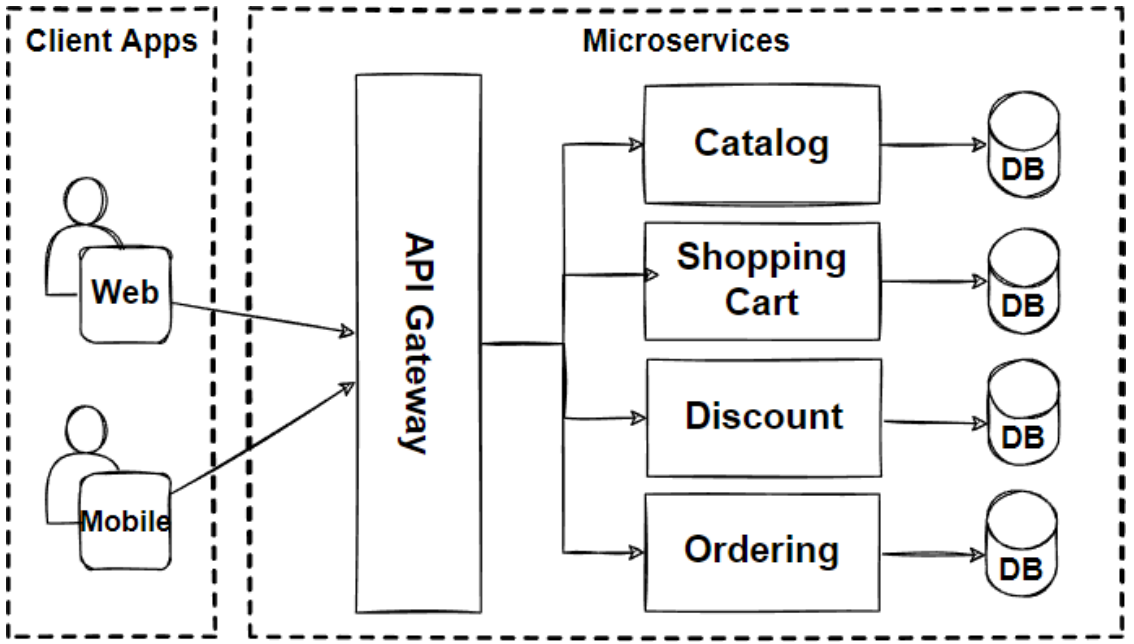
- From Martin Fowlers Microservices article;
- Componentization via Services
- Organized around Business Capabilities
- Products not Projects
- Smart endpoints and dumb pipes
- Decentralized Governance
- Decentralized Data Management
- Infrastructure Automation
- Design for failure



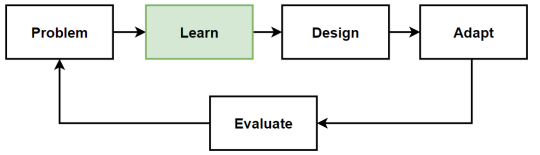
Benefits of Microservices Architecture



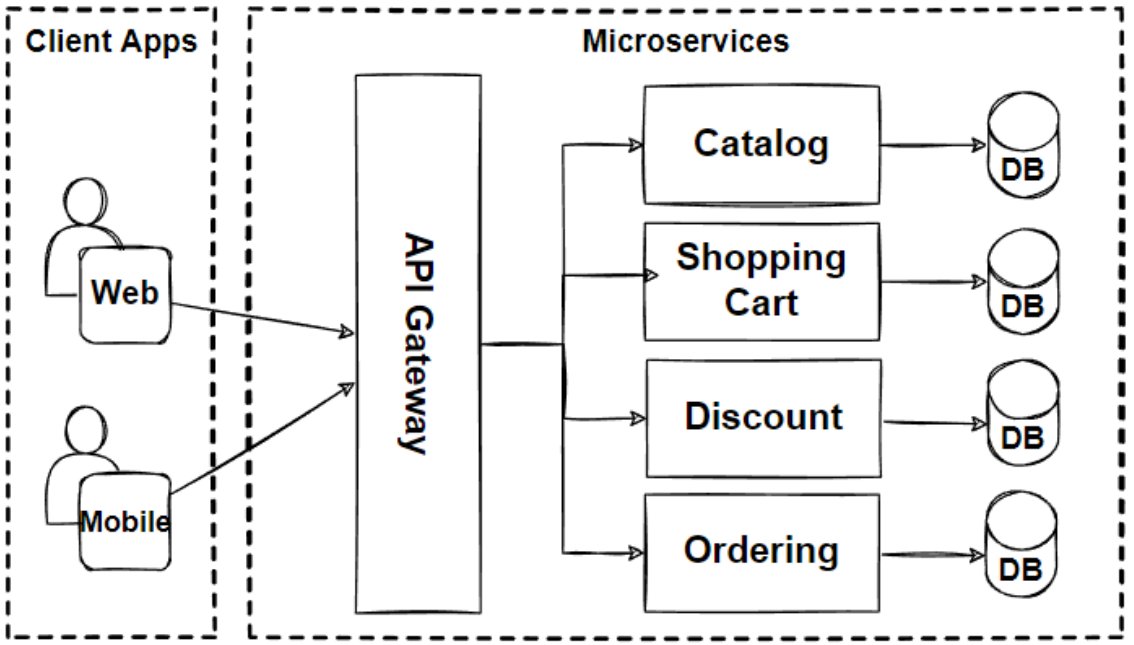
- **Agility, Innovation and Time-to-market**
  Microservices architectures make applications easier to scale and faster to develop, enabling innovation and accelerating time-to-market for new features.
- **Flexible Scalability**
  Microservices can be scaled independently, so you scale out sub-services that require less resources, without scaling out the entire application.
- **Small, focused teams**
  Microservices should be small enough that a single feature team can build, test, and deploy it.
- **Small and separated code base**
  Microservices are not sharing code or data stores with other services, it minimizes dependencies, and that makes easier to adding new features.



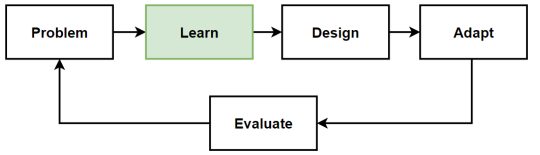
Benefits of Microservices Architecture -2



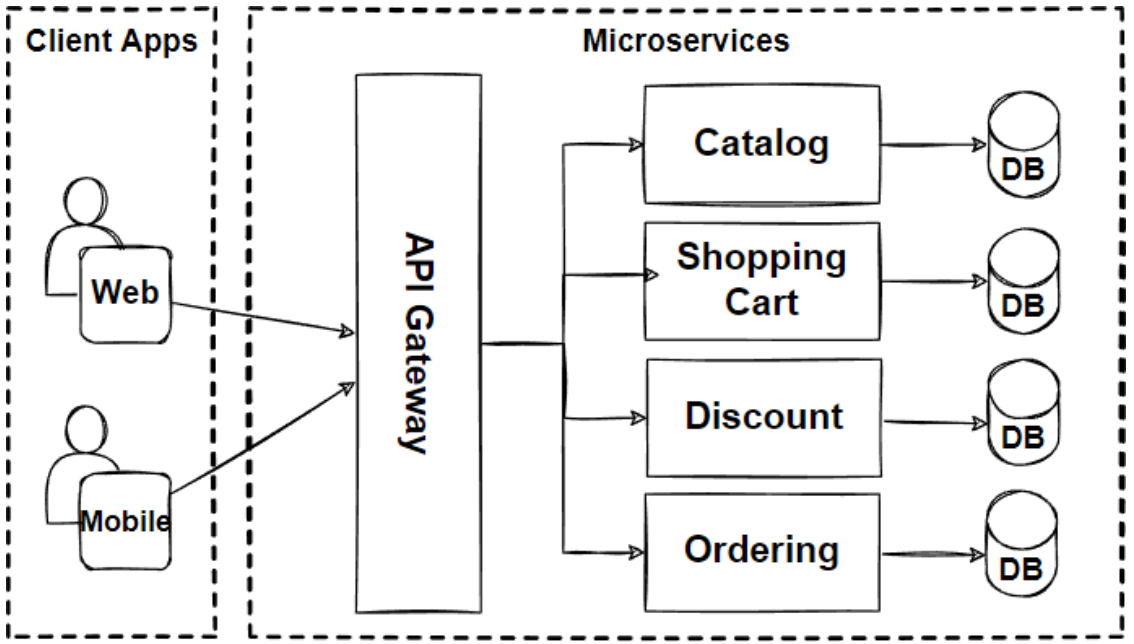
- **Easy Deployment**
  Microservices enable continuous integration and continuous delivery, making it easy to try out new ideas and to roll back if something doesn't work.
- **Technology agnostic, Right tool for the job**
  Small teams can pick the technology that best fits their microservice and using a mix of technology stacks on their services.
- **Resilience and Fault isolation**
  Microservices are fault toleranced and handle faults correctly for example by implementing retry and circuit breaking patterns.
- **Data isolation**
  Databases are separated with each other according to microservices design. Easier to perform schema updates, because only a single database is affected.



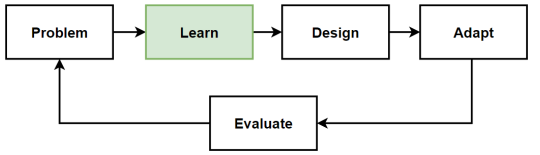
Challenges of Microservices Architecture



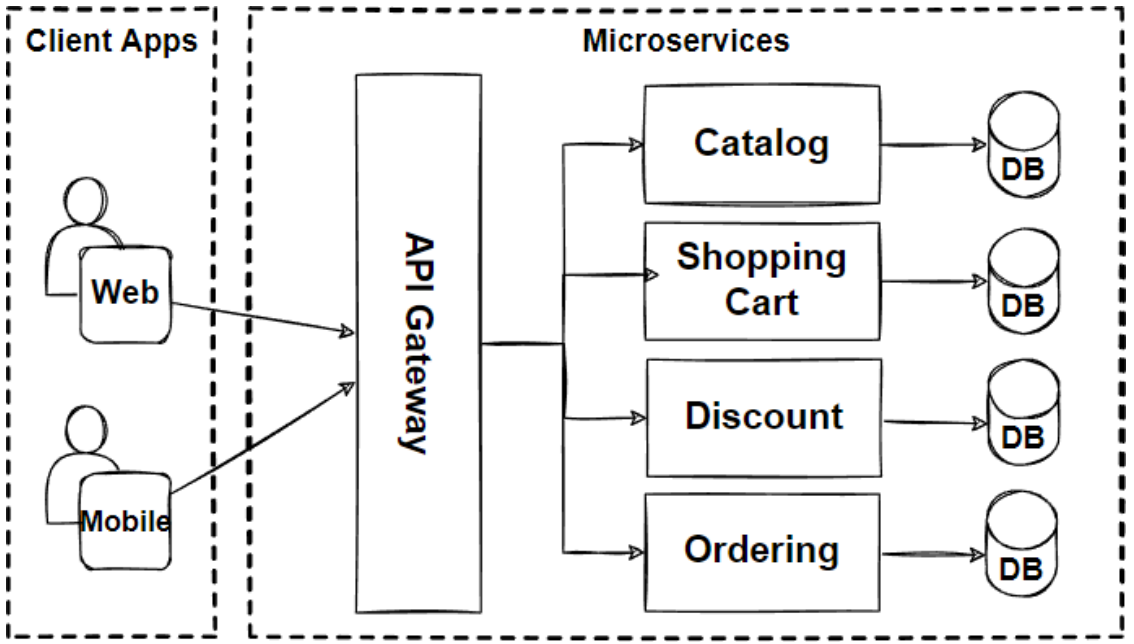
- **Complexity**
  Each service is simpler, but the entire system is more complex. Deployments and Communications can be complicated for hundreds of microservices.
- **Network problems and latency**
  Microservice communicate with inter-service communication, we should manage network problems. Chain of services increase latency problems and become chatty API calls.
- **Development and testing**
  Hard to develop and testing these E2E processes in microservices architectures if we compare to monolithic ones.
- **Data integrity**
  Microservice has its own data persistence. Data consistency can be a challenge. Follow eventual consistency where possible.



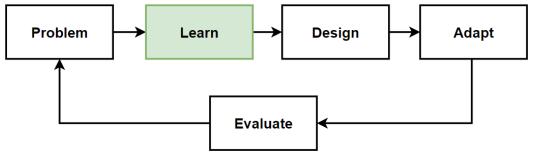
Challenges of Microservices Architecture -2



- **Deployment**
  Deployments are challenging. Require to invest in quite a lot of devops automation processes and tools. The complexity of microservices becomes overwhelming for human deployment.
- **Logging & Monitoring**
  Distributed systems are required to centralized logs to bring everything together. Centralized view of the system to monitor sources of problems.
- **Debugging**
  Debugging through local IDE isn't an option anymore. It won't work across dozens or hundreds of services.

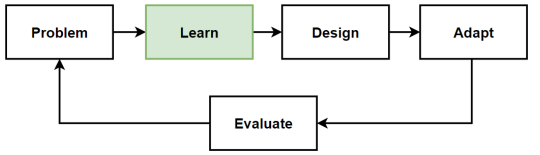


When to Use Microservices Architecture

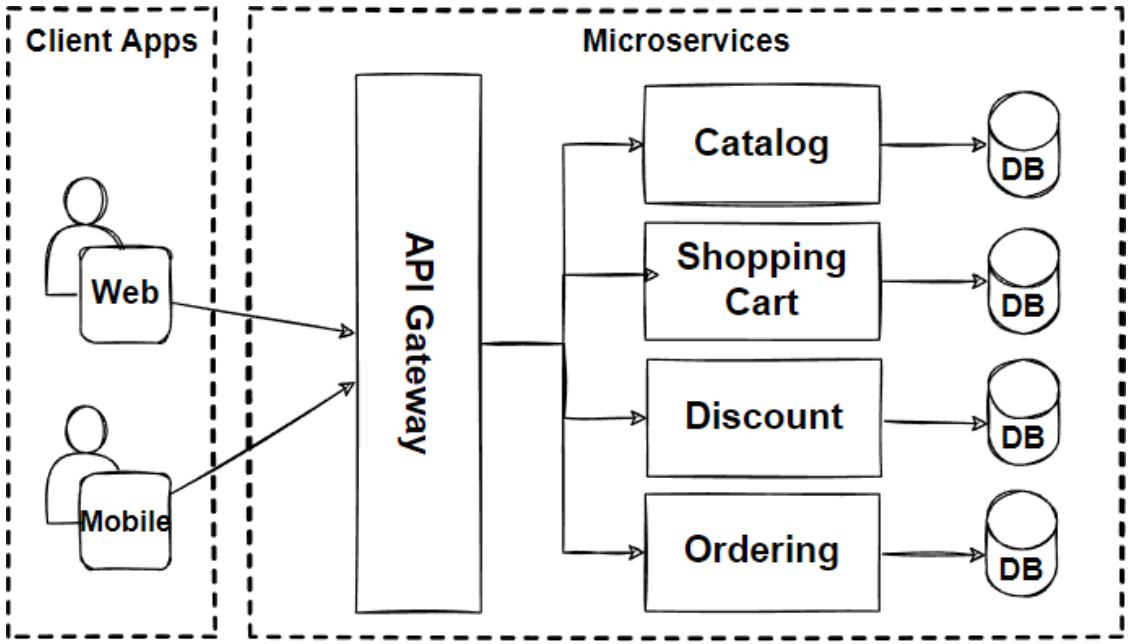


- **Make Sure You Have a "Really Good Reason" for Implementing Microservices**
  Check if your application can do without microservices. When your application requires agility to time-to-market with zero-down time deployments and updated independently that needs more flexibility.
- **Iterate With Small Changes and Keep the Single-Process Monolith as Your "Default"**
  Sam Newman and Martin Fowler offers Monolithic-First approach. Single-process monolithic application comes with simple deployment topology. Iterate and refactor with turning a single module from the monolith into a microservices one by one.
- **Required to Independently Deploy New Functionality with Zero Downtime**
  When an organization needs to make a change to functionality and deploy that functionality without affecting rest of the system.
- **Required to Independently Scale a Portion of Application**
  Microservice has its own data persistence. Data consistency can be a challenge. Follow eventual consistency where possible.

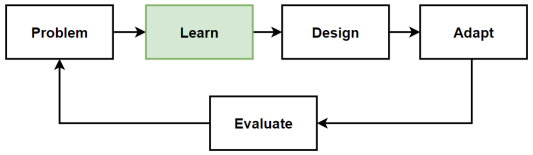
When to Use Microservices Architecture -2



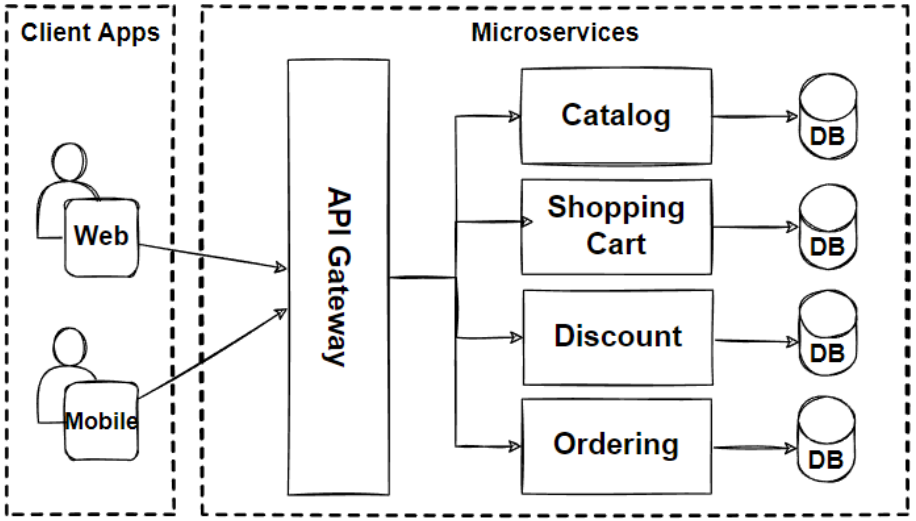
- **Data Partitioning with different Database Technologies**
  Microservices are extremely useful when an organization needs to store and scale data with different use cases. Teams can choose the appropriate technology for the services they will develop over time.
- **Autonomous Teams with Organizational Upgrade**
  Microservices will help to evolve and upgrade your teams and organizations. Organizations need to distribute responsibility into teams, where each team makes decisions and develops software autonomously.



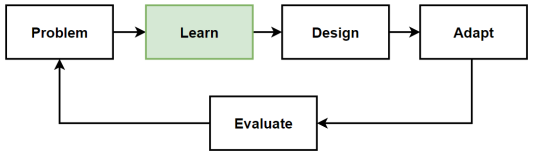
When Not to Use Microservices
Anti-Patterns of Microservices



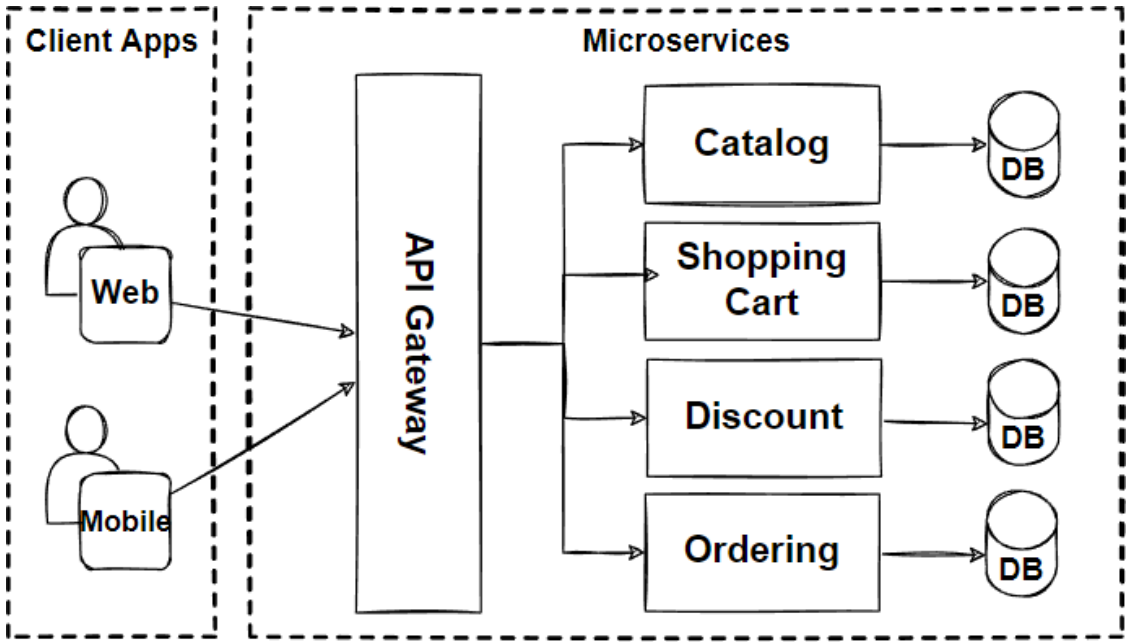
- **Don’t do Distributed Monolith**
  Make sure that you decompose your services properly and respecting the decoupling rule like applying bounded context and business capabilities principles.
- Distributed Monolith is the worst case because you increase complexity of your architecture without getting any benefit of microservices.
- **Don’t do microservices without DevOps or cloud services**
  Microservices are embrace the distributed cloud-native approaches. And you can only maximize benefits of microservices with following these cloud-native principles.
  - CI/CD pipeline with devops automations
  - Proper deployment and monitoring tools
  - Managed cloud services to support your infrastructure
  - Key enabling technologies and tools like Containers, Docker, and Kubernetes
  - Following asnyc communications using Messaging and event streaming services



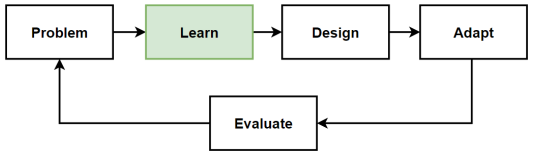
When Not to Use Microservices



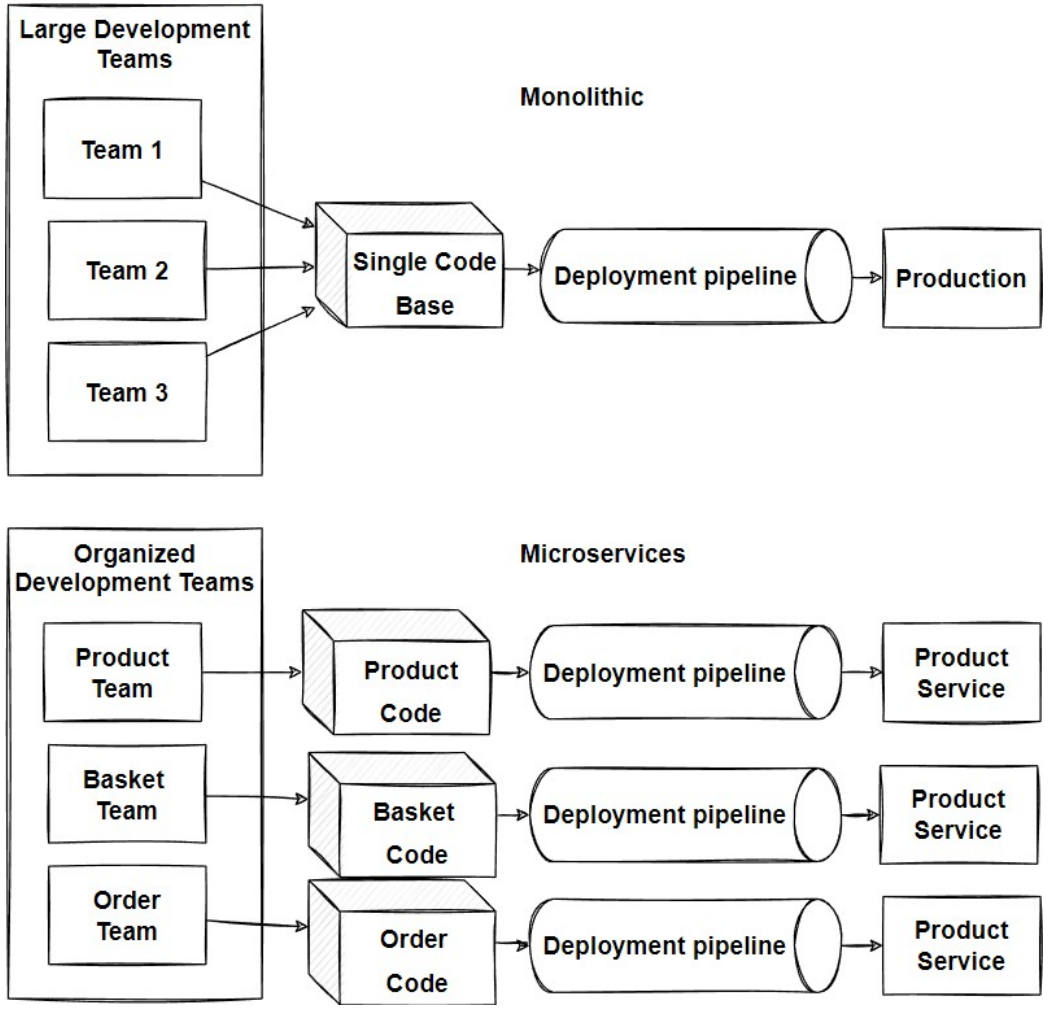
- **Limited Team sizes, Small Teams**
  If you don't have a team size that cannot handle the microservice workloads, This will only result in the delay of delivery.
- For a small team, a microservice architecture can be hard to justify, because team is required just to handle the deployment and management of the microservices themselves.
- **Brand new products or startups**
  If you are working on a new startup or brand new product which require significant change when developing and iterating your product, then you should not start with microservices.
- Microservices are so expensive when you re-design your business domains. Even if you do become successful enough to require a highly scalable architecture.
- **The Shared Database anti-pattern**



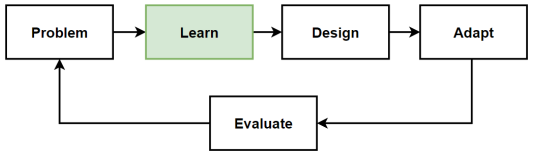
Monolithic vs Microservices Architecture Comparison

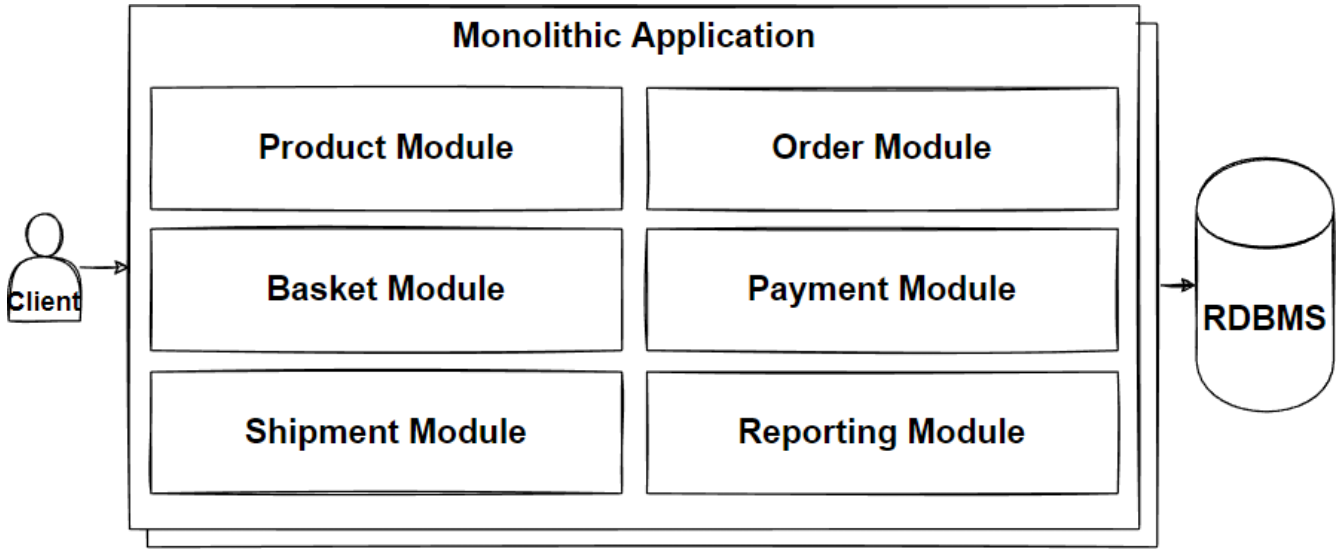


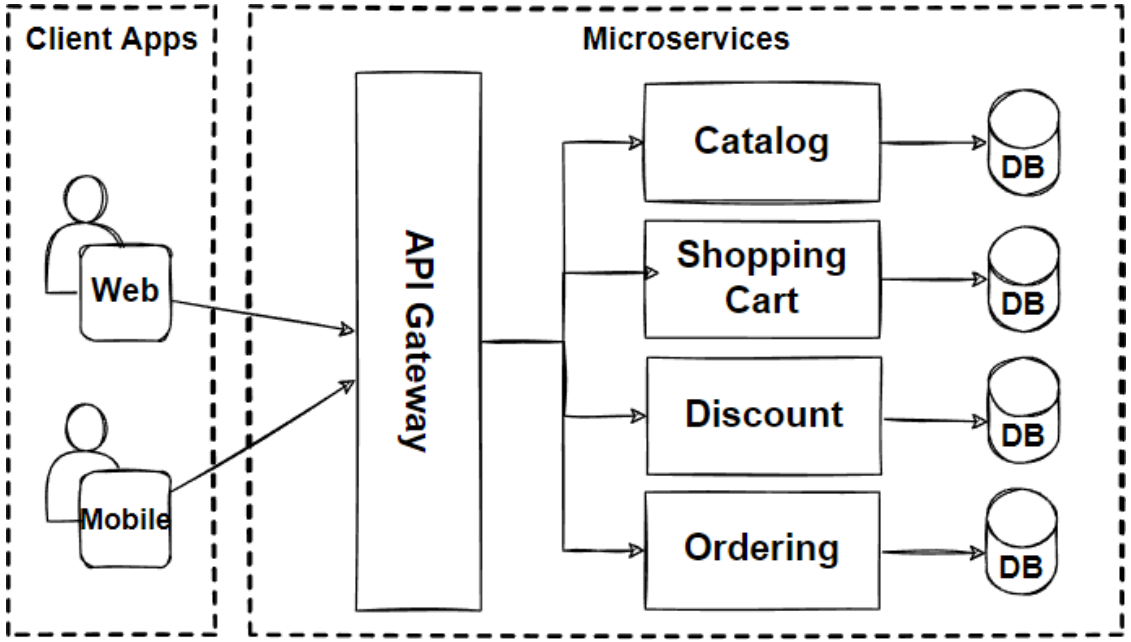
- **Application Architecture**
  Monolith has a simple straightforward structure of one undivided unit. Microservices have a complex structure that consists of various heterogeneous services and databases.
- **Scalability**
  Monolithic application is scaled as a whole single unit, but microservices can be scaled unevenly. encourages companies to migrate their applications to microservices.
- **Deployment**
  Monolithic application provides fast and easy deployment of the whole system. Microservices provides zero-downtime deployment and CI/CD automation.
- **Development team**
  If your team doesn't have experience with microservices and container systems, building a microservices-based application will be difficult.



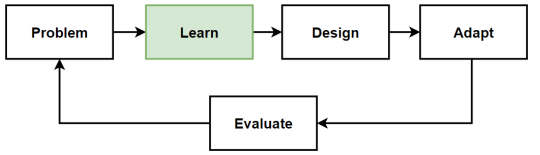
Architecture Comparison



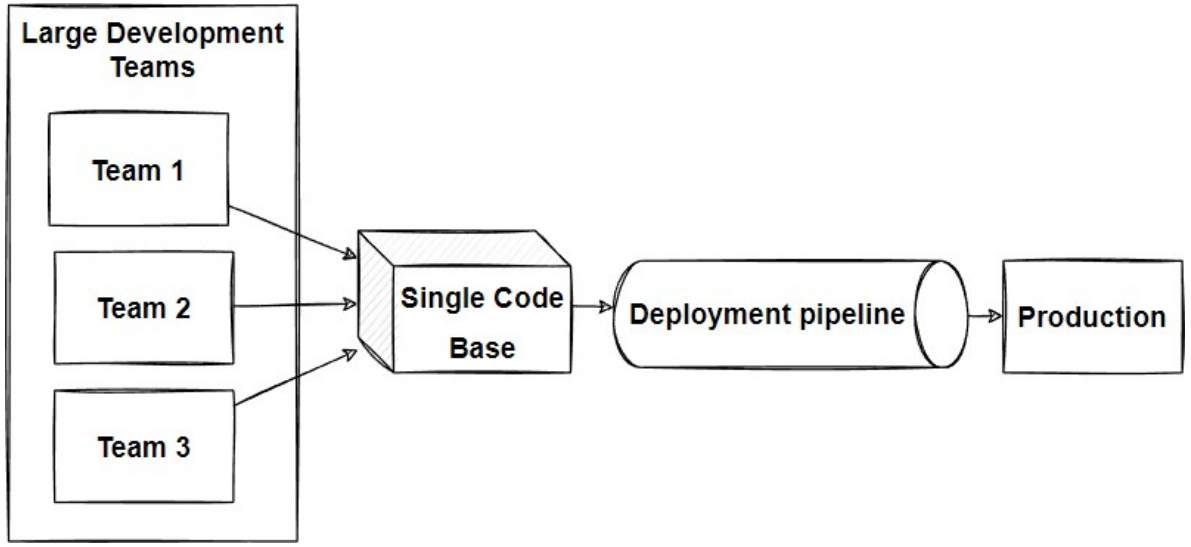




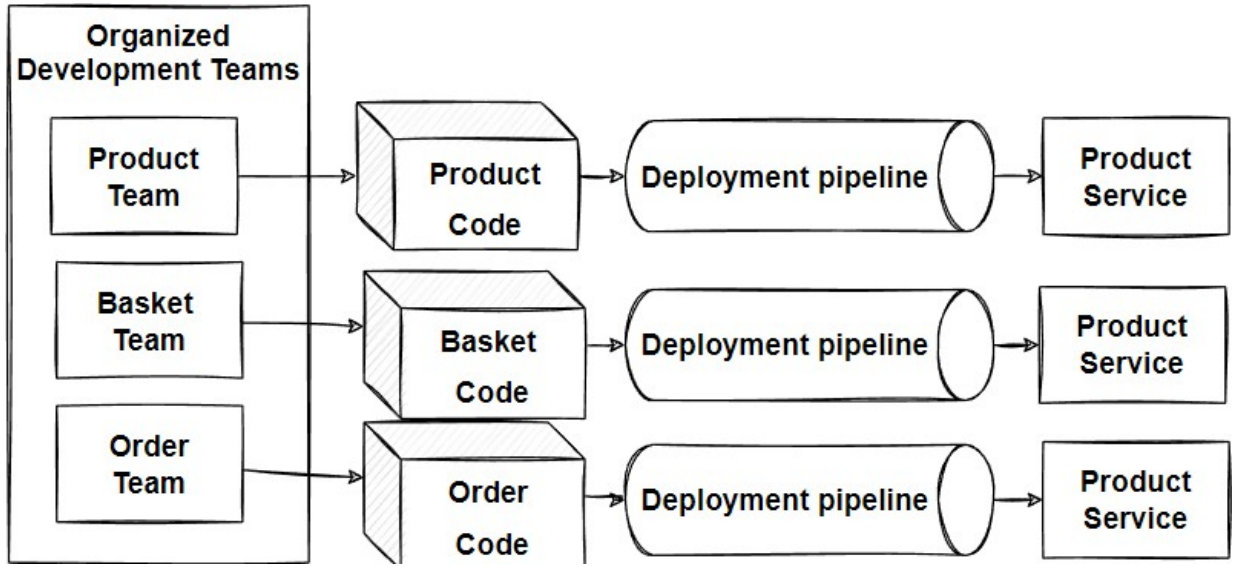
Deployment Comparison



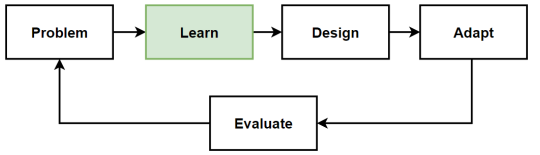
Monolithic



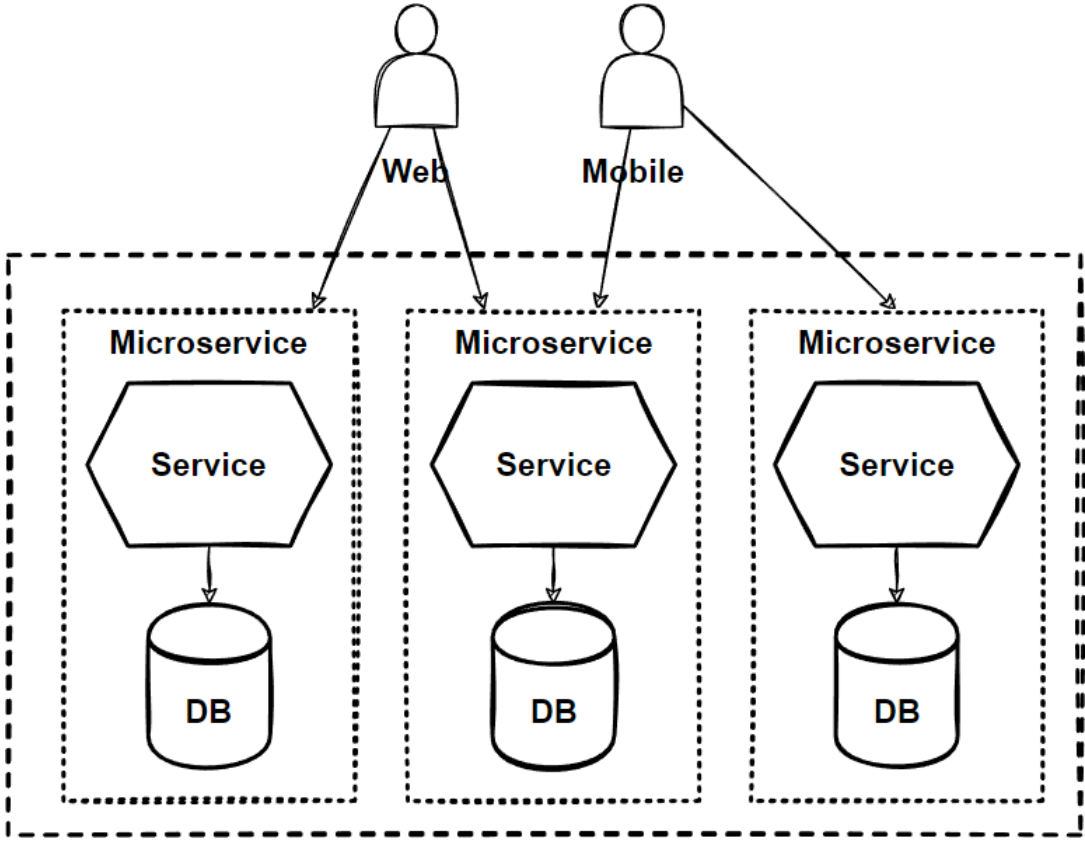
Microservices



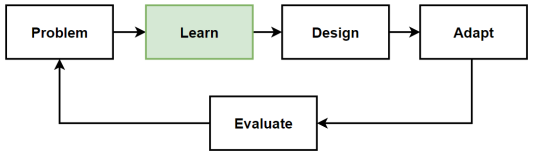
The Database-per-Service Pattern



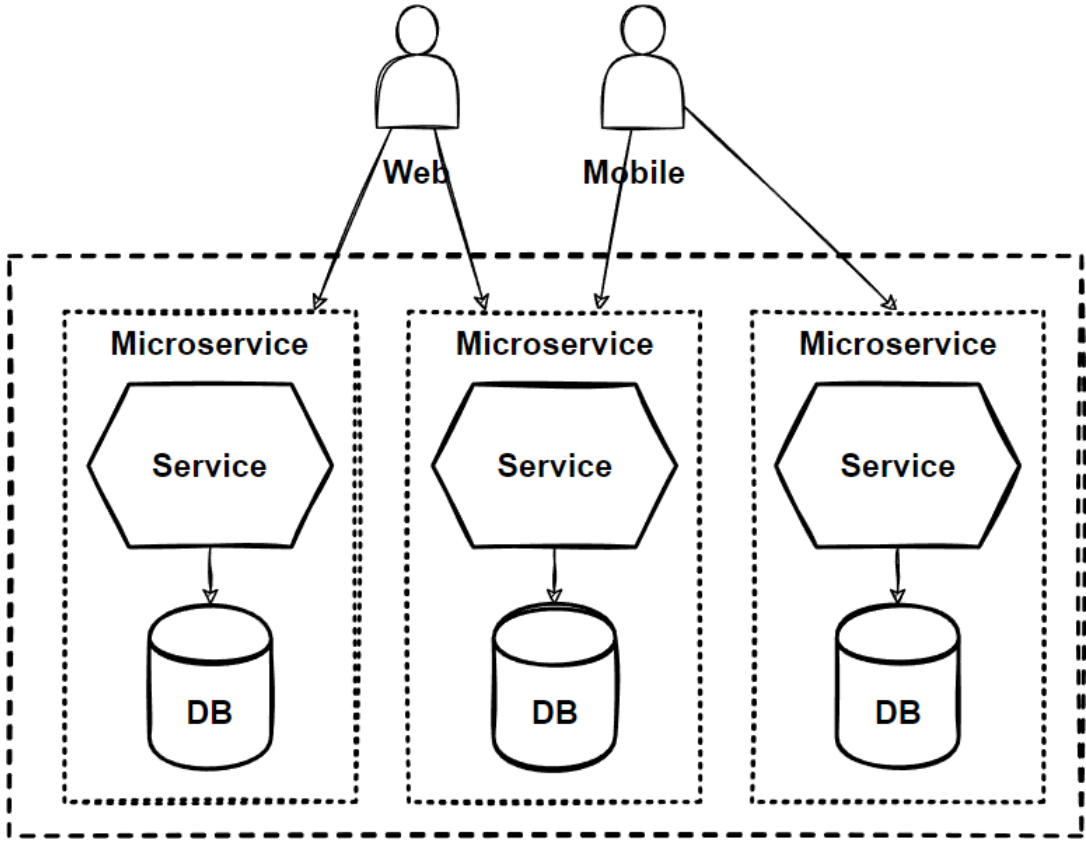
- Core characteristic of the microservices architecture is the **loose coupling of services**. every service should have its **own databases**, it can be **polyglot persistence** among to microservices.
- E-commerce application. We will have **Product - Ordering** and **SC** microservices that **each services data in their own databases**. Any changes to one database **don't impact other microservices.**
- The **service's database can't be accessed** directly by other microservices. Each service's persistent data can only be accessed via **Rest APIs.**



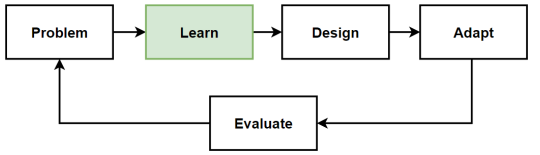
Benefits of the Database-per-Service Pattern with Polygot Persistence



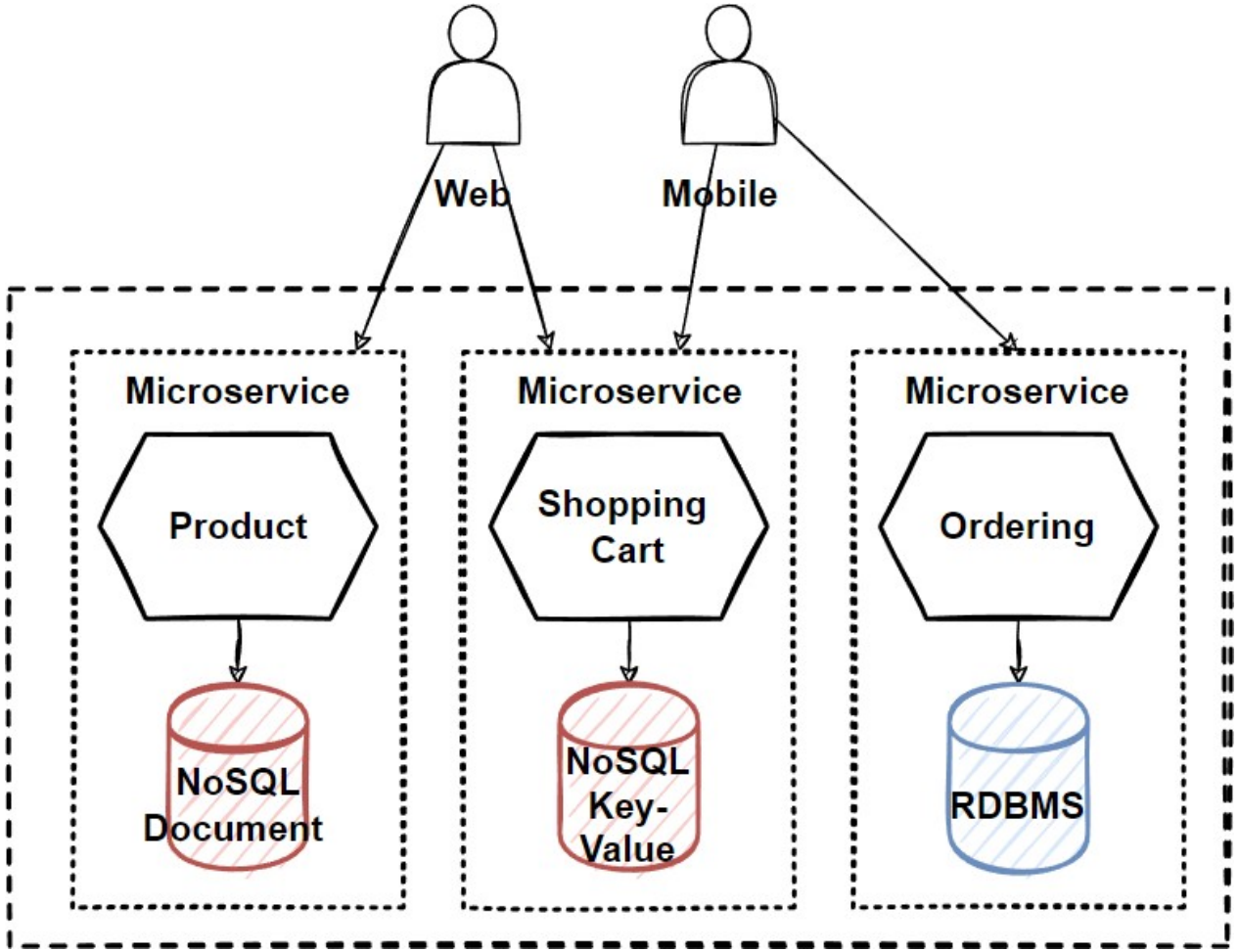
- **Data schema changes** made easy without impacting other microservices.
- Each **database** can **scale independently.**
- Microservices domain data is **encapsulated** within the service.
- If one of the database server is down, this will **not affect to other services.**
- **Polyglot data persistence** gives ability to select the **best optimized storage** needs per microservices.



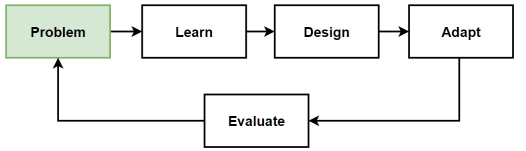
E-Commerce with Database-per-Service Pattern and Polygot Persistence



- Product service using **NoSQL document database** for storing catalog related data.
- Shopping cart service using a **distributed cache** that supports its simple, **key-value data store**.
- Ordering service using a **relational database** to handle the rich relational structure.
- **NoSQL databases** able to massive scale and high availability, and also schemaless structure give flexibility.



Problem: Scale and Deploy Independently

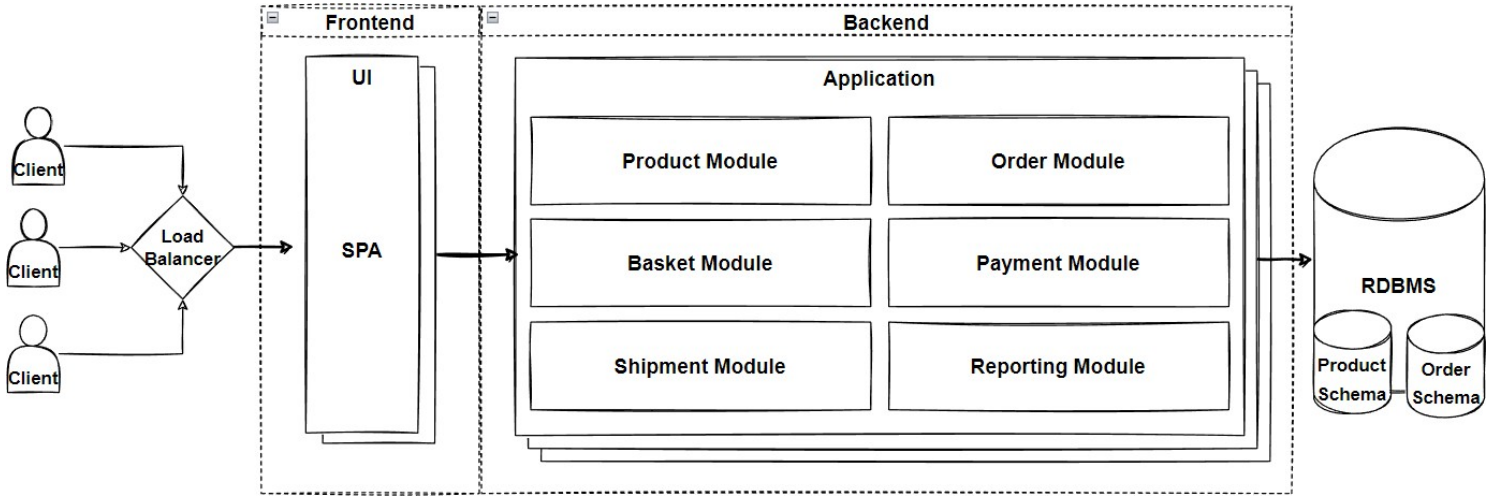


Problems

- Our E-Commerce Business is growing
- Business teams are **separated teams** as per departments; Product, Sale, Payment.
- Teams wants to agile and **add new features immediately** to compete the market
- Innovate and experiment with new features as soon as possible
- **Deploy features immediately**, not waiting for deployment dates
- **Flexible scale** for **market peek times** like blackfriday sales
- Handle and process **millions of request** in a acceptable latency with better performance.
- Required not only technology change but also **organizational change is mandatory**.

Solutions

- Microservices Architecture



Before Design – What we have in our design toolbox ?

Architectures

- **Microservices Architecture**

Patterns&Principles

- **The Database-per-Service Pattern**
- **Polygot Persistence**

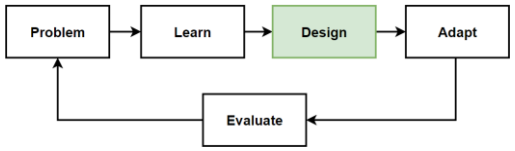
Non-FR

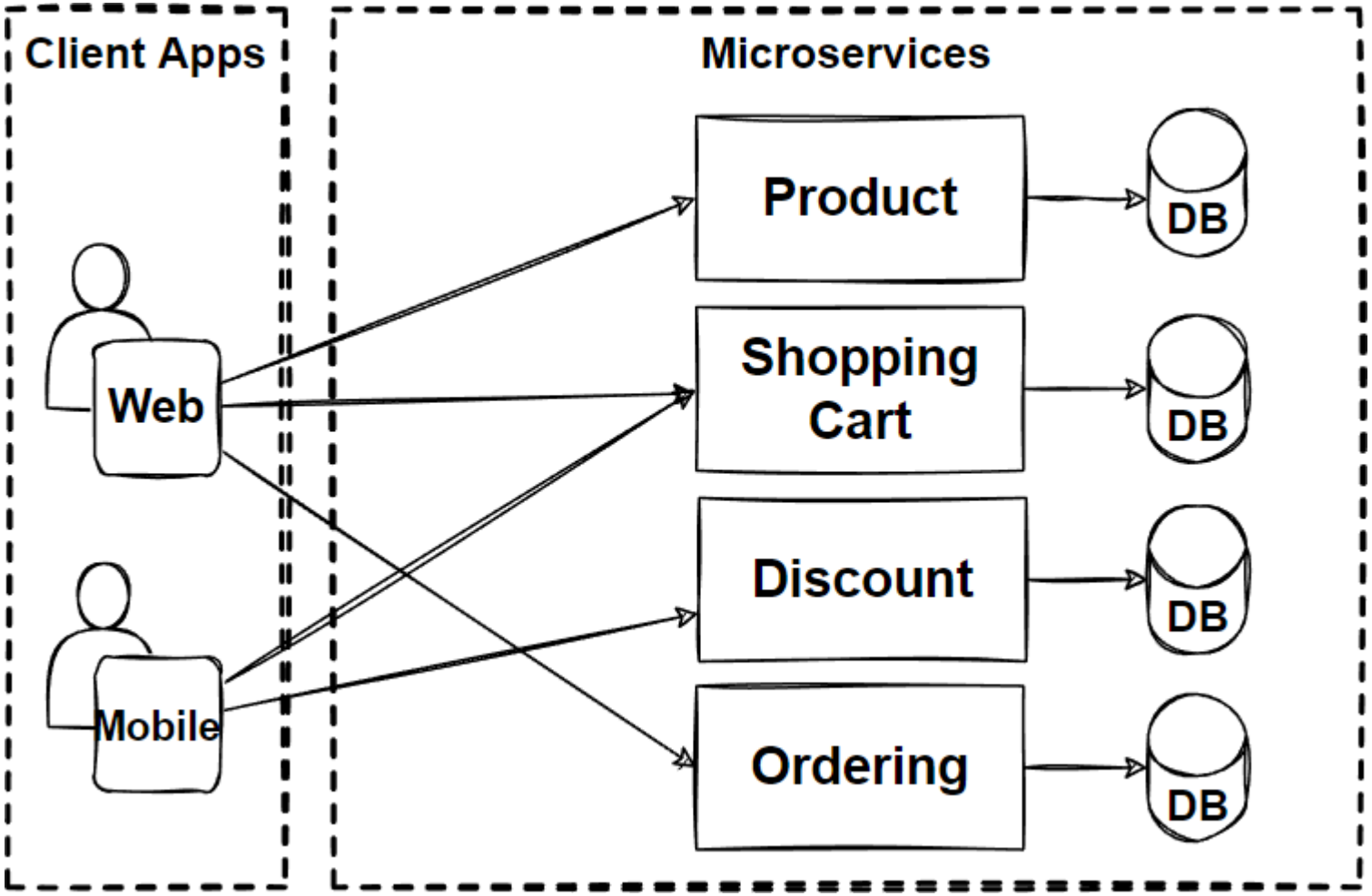
- **High Scalability**
- **High Availability**
- **Millions of Concurrent User**
- **Independent Deployable**
- **Technology agnostic**
- **Data isolation**
- **Resilience and Fault isolation**

FR

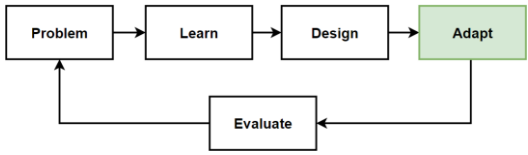
- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

Design: Microservices Architecture





Adapt: Microservices Architecture



Frontend SPAs

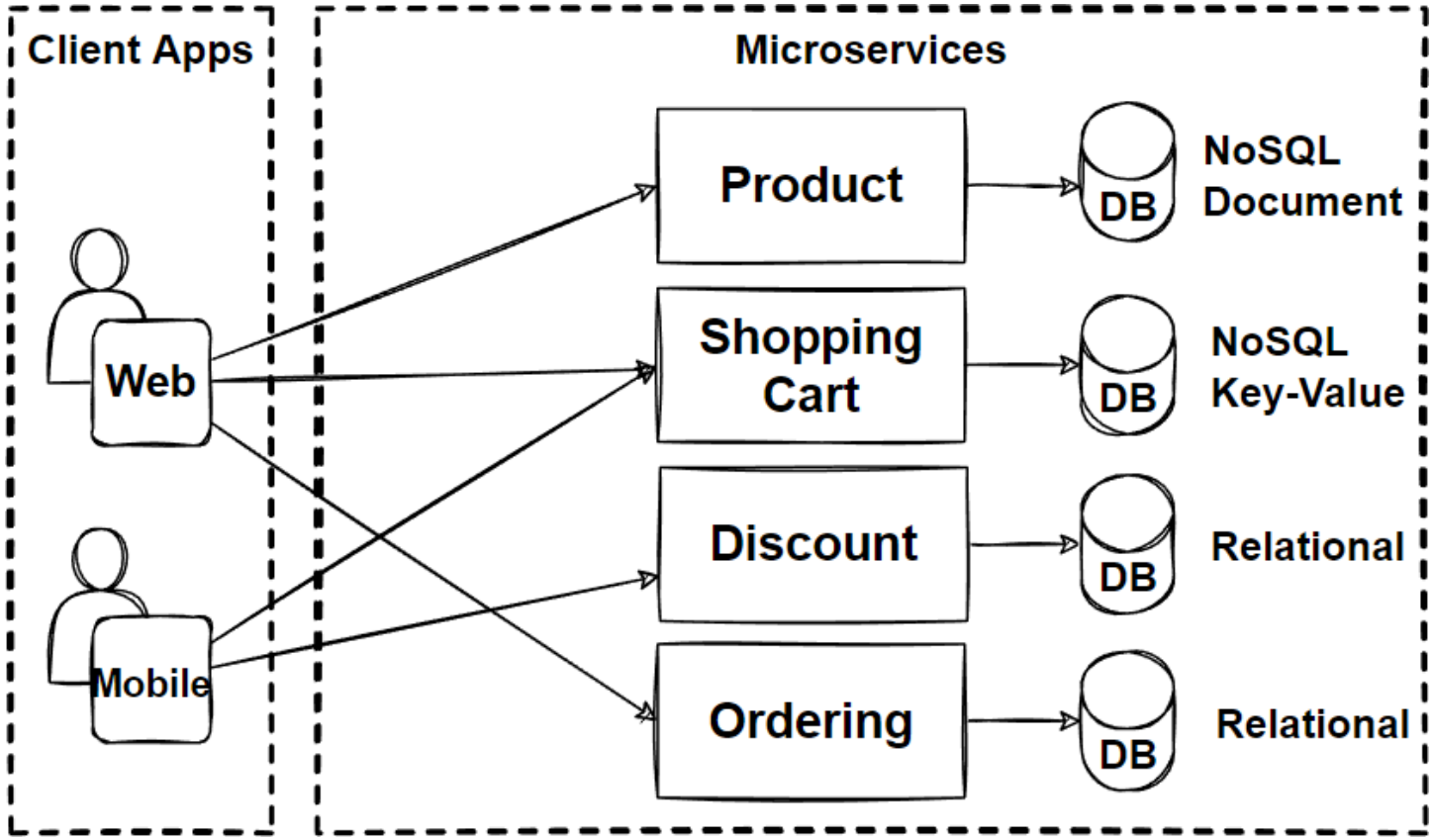
- Angular
- Vue
- React

Backend Microservices

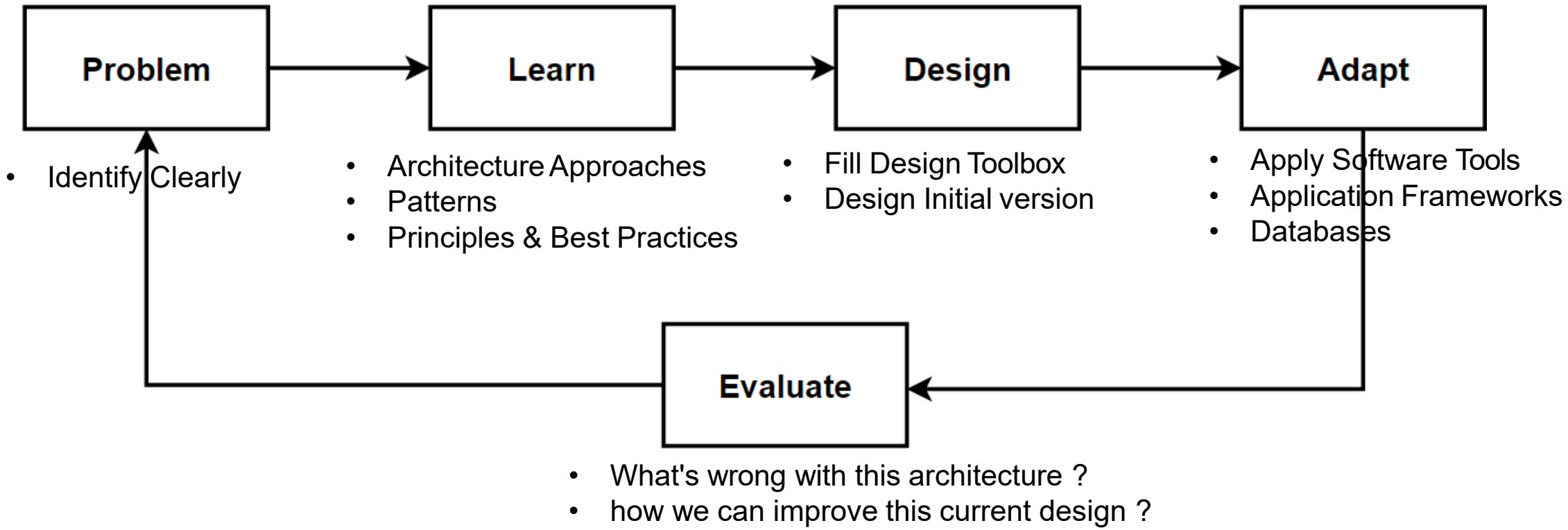
- Java – Spring Boot
- .Net – Asp.net
- JS – NodeJS
- Python – Django, Flask

Database

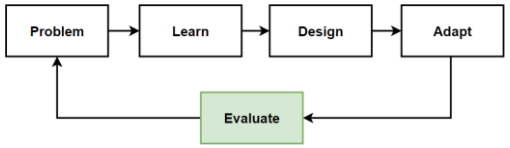
- MongoDB – NoSQL Document DB
- Redis – NoSQL Key-Value Pair
- Postgres – Relational
- SQL Server – Relational



Way of Learning – The Course Flow



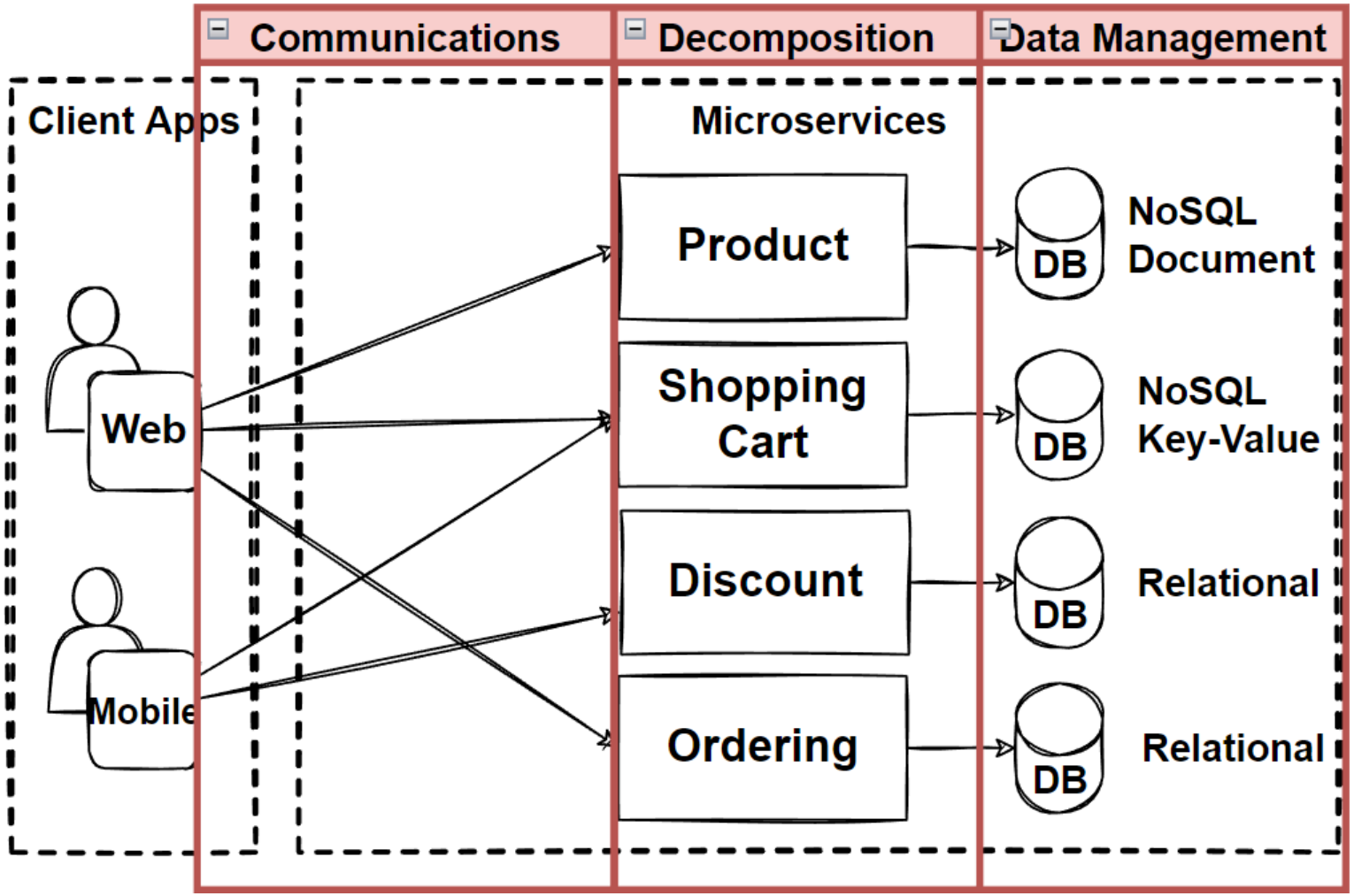
Evaluate: Microservices Architecture



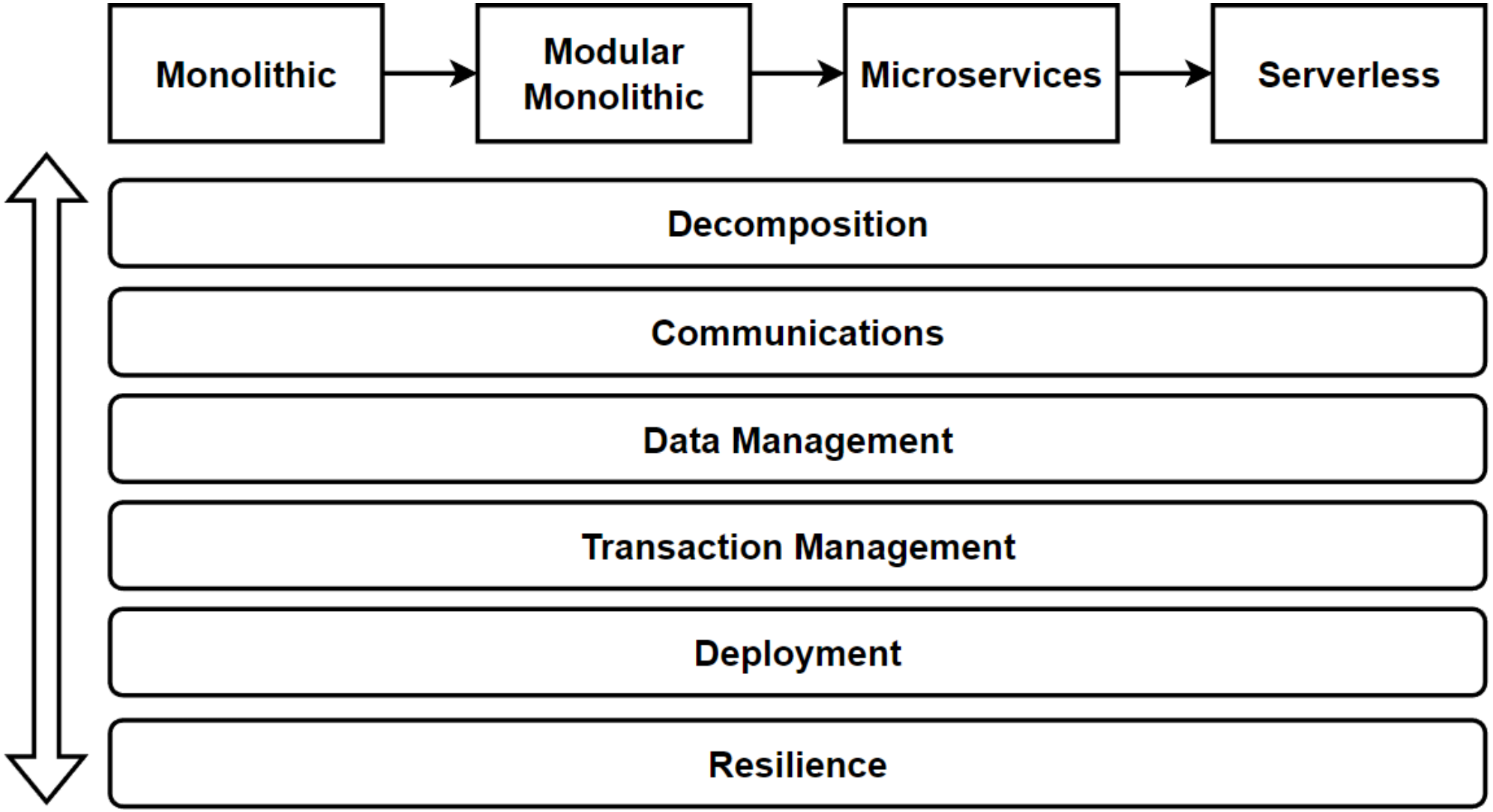
Main Considerations

- Decomposition – Breaking Down Services
- Communications
- Data Management

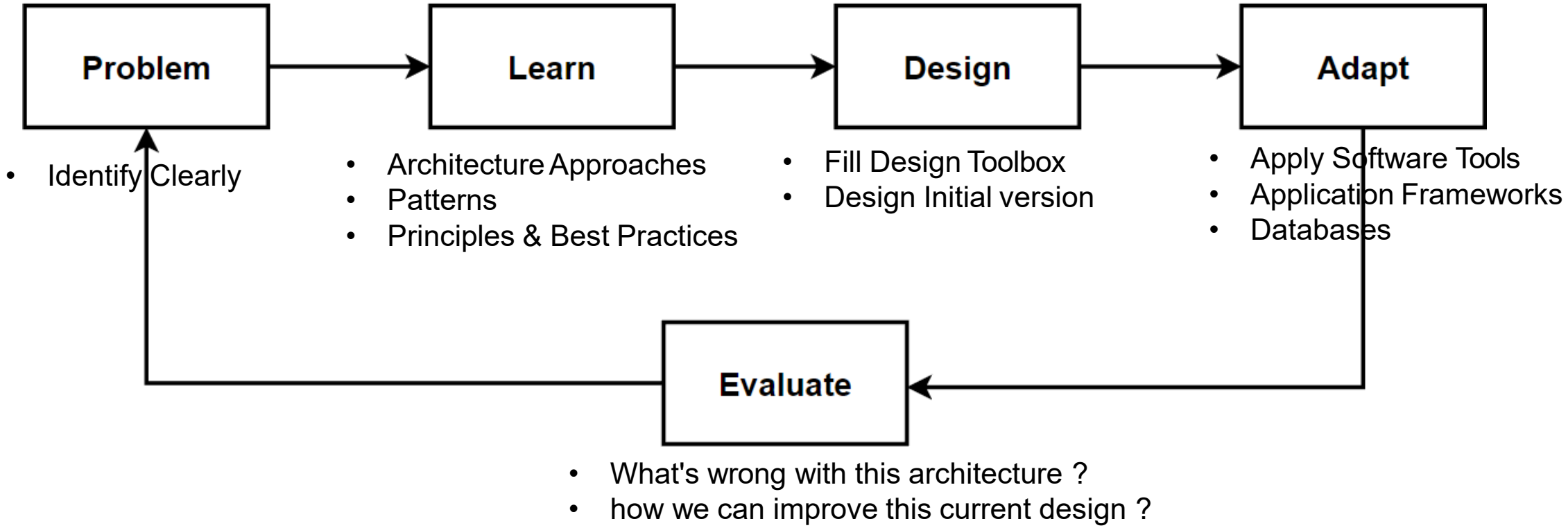
- Transaction Management
- Deployments
- Resilience



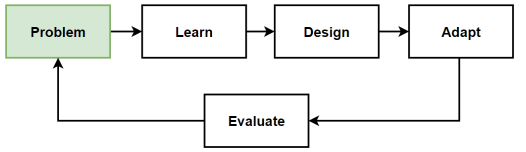
Architecture Design – Vertical Considerations



Way of Learning – The Course Flow



Problem: Break Down Application into Microservices

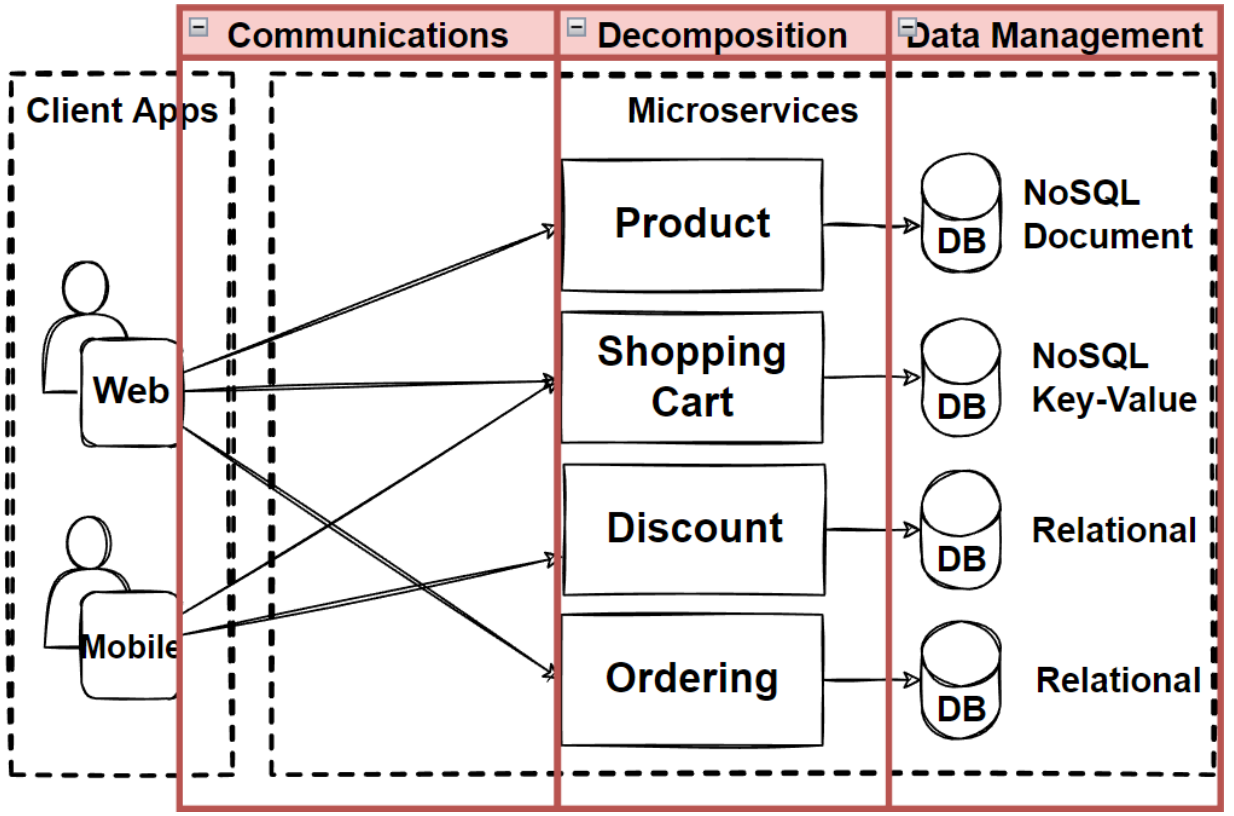


Problems

- Our E-Commerce Business is growing
- Teams wants to agile and add new features immediately to compete the market
- Required Independent Scale and Deployments
- We should clearly identify microservices which parts could be independent scale and deploy

Solutions

- Microservices Decomposition Patterns



Decomposition of Microservices Architecture

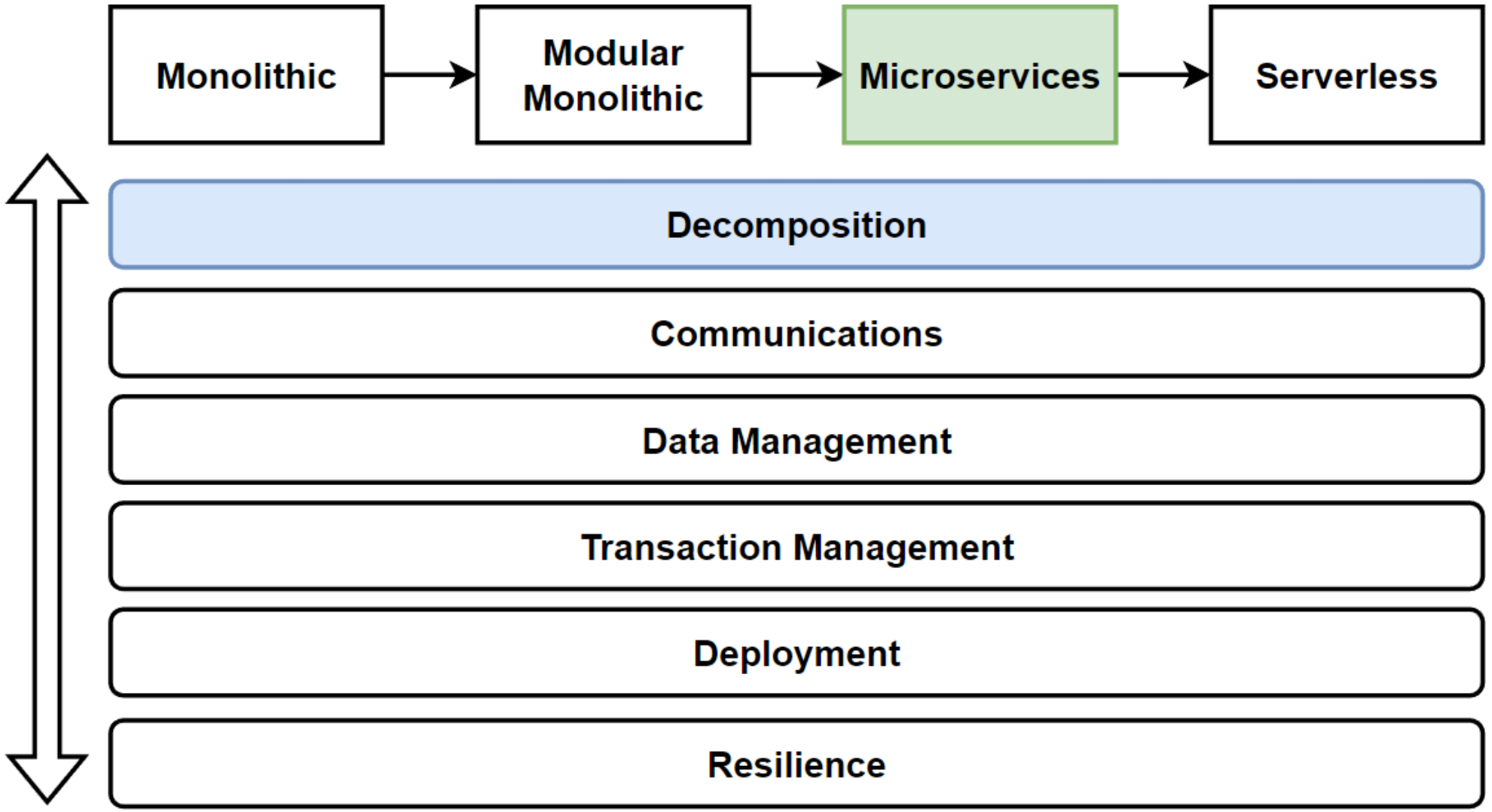
Breaking Down Application into Microservices

Microservices Decomposition Patterns

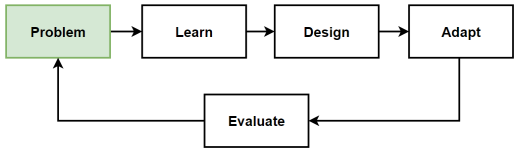
Decomposing Applications for Independent Deployability and Scalability

Breaking Down our E-Commerce application with Microservices Decomposition Patterns

Architecture Design – Vertical Considerations



Problem: Break Down Application into Microservices

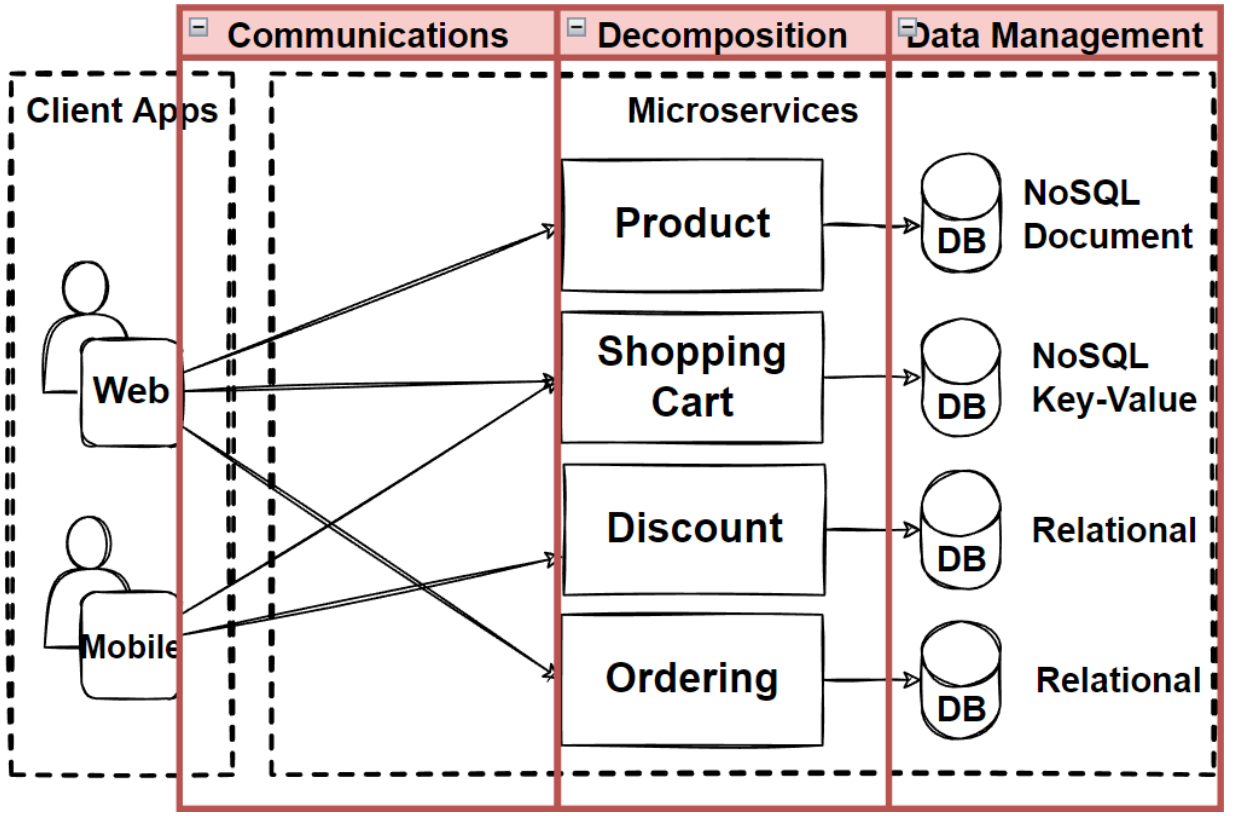


Problems

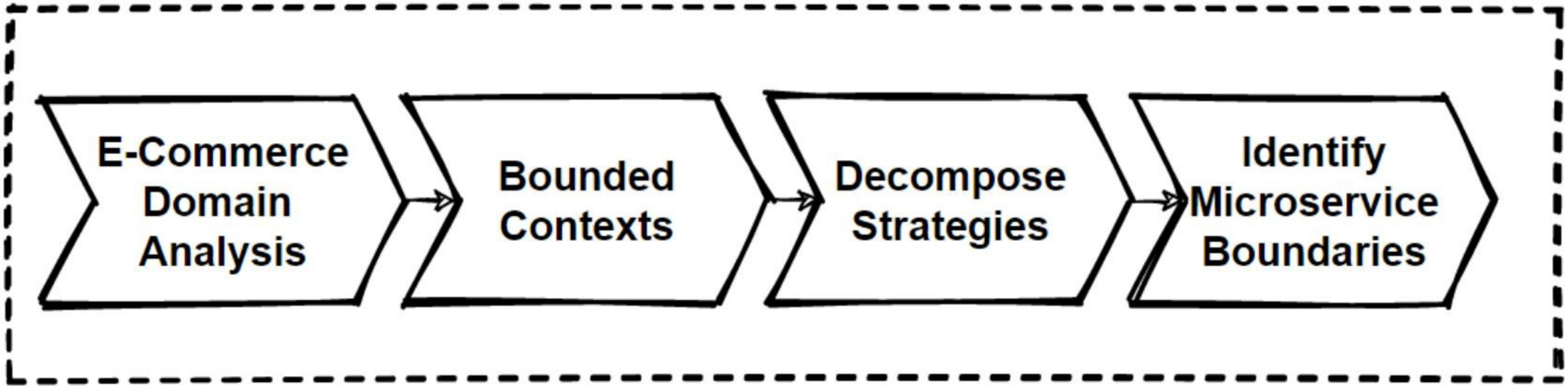
- Our E-Commerce Business is growing
- Teams wants to agile and add new features immediately to compete the market
- **Required Independent Scale and Deployments**
- We should clearly identify microservices which parts could be independent scale and deploy

Solutions

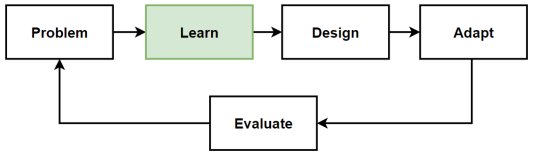
- **Microservices Decomposition Patterns**



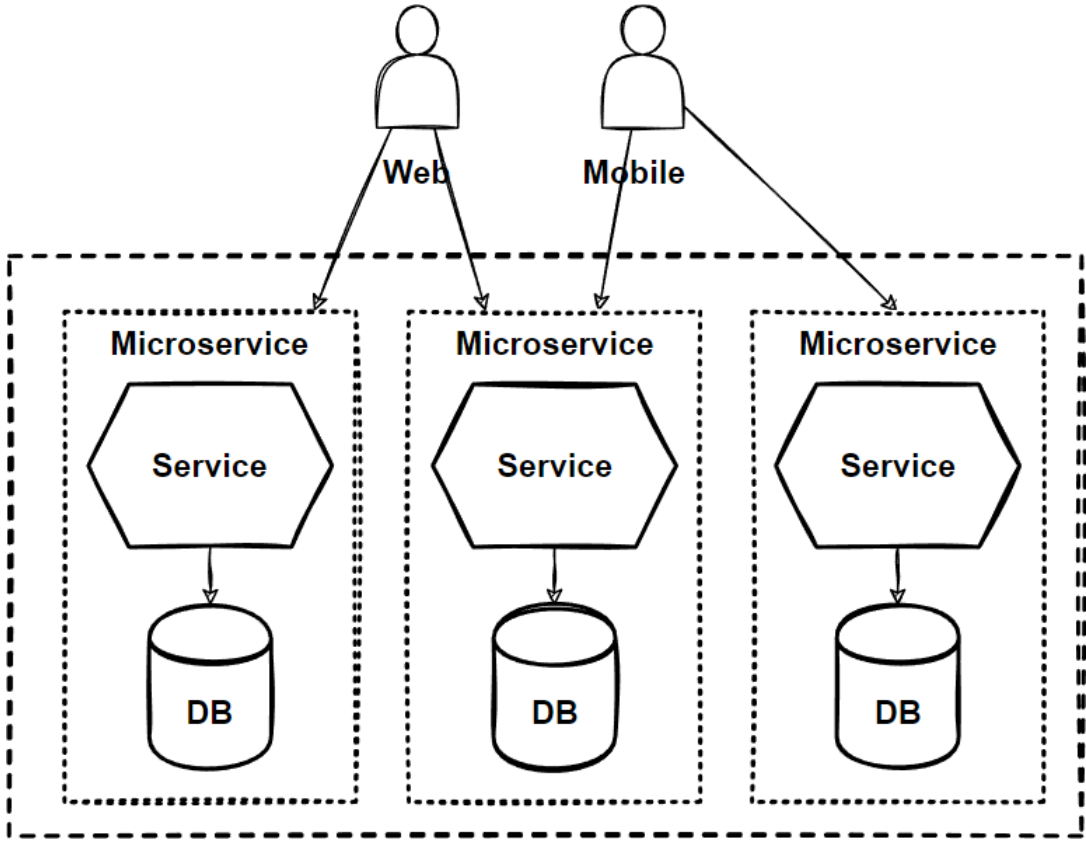
Microservices Decomposition Path



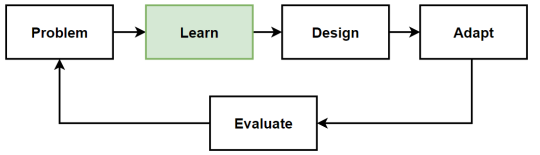
Why we need to Decompose ?



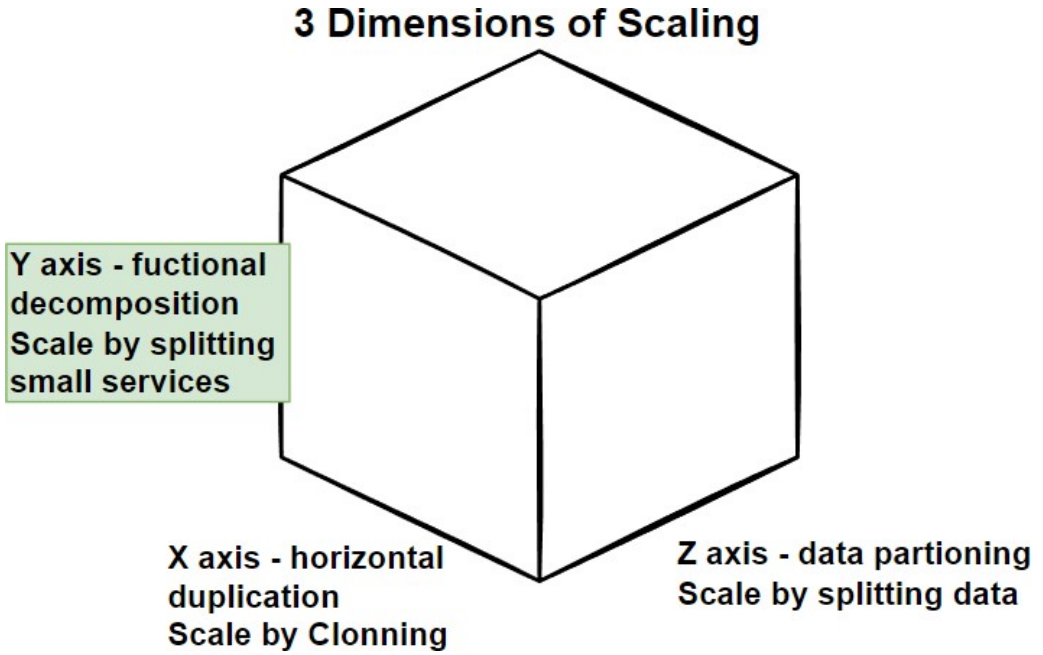
- What is the **main reason** behind the **decomposition** of microservices ? provide to **Scale Independently.**
- Main benefits of microservices are "**Independent Scale** and **Deployments**".
- **Clearly identify microservices** which parts could be required independent scale and deploy.
- **Design Principle: Decompose services by scalability requirements**
- Applications are consist of multiple modules or services, with **different requirements for scaling**.
- Example: Our **e-commerce application** have a public-facing website and a separate administration website.
- 2 services required **different scalability requirements**.



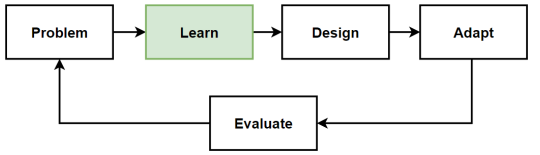
The Scale Cube



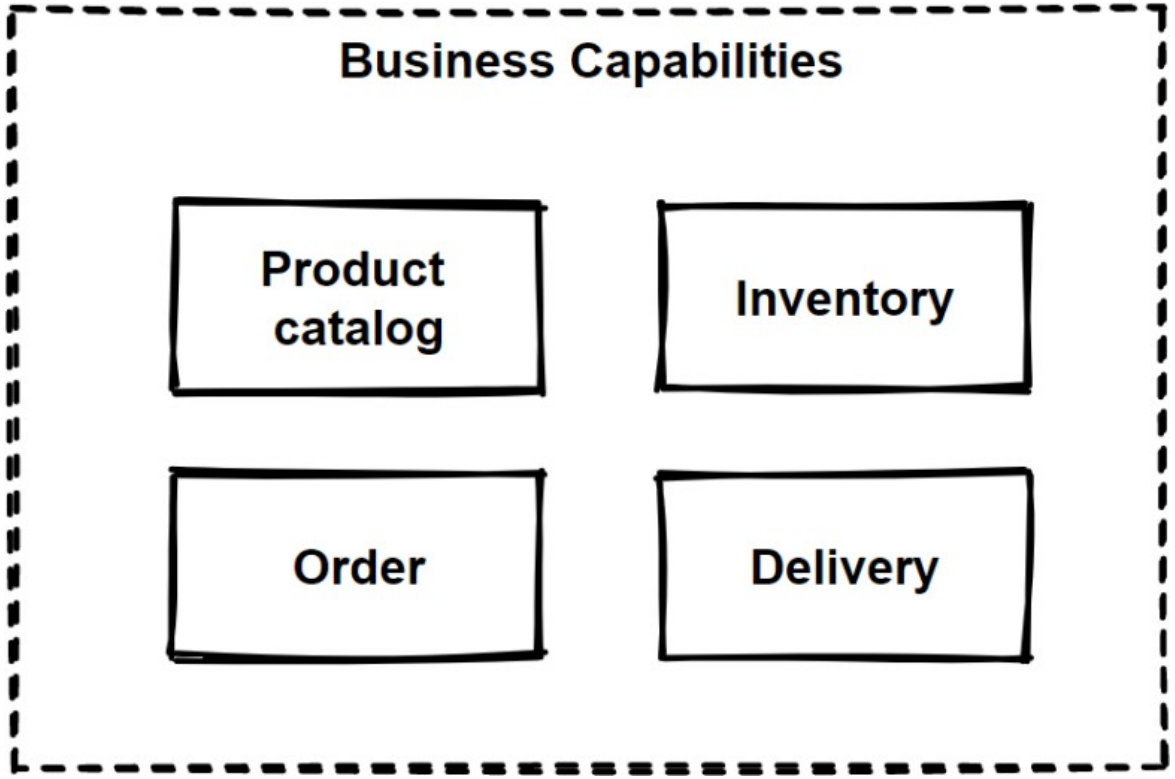
- **X-Axis:** Horizontal Duplication and Cloning of services and data
- **Y-Axis:** Functional Decomposition and Segmentation – Microservices
- **Z-Axis:** Service and Data Partitioning along Customer Boundaries - Shards/Pods
- **Functional decomposition** can be achieved by decoupling your architecture into functions **with Y-axis.**
- **Microservices** is an example for **functional decomposing**.
- **Y-axis** scaling splits the application into multiple, different services.
- **Combining** both **X** and **Y-Axis** scaling with microservices architecture can give **better scalability**.



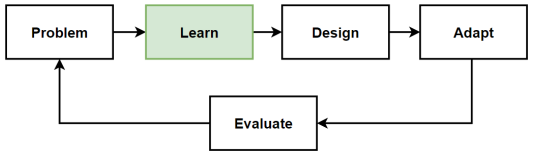
Microservices Decomposition Pattern – Decompose by Business Capability



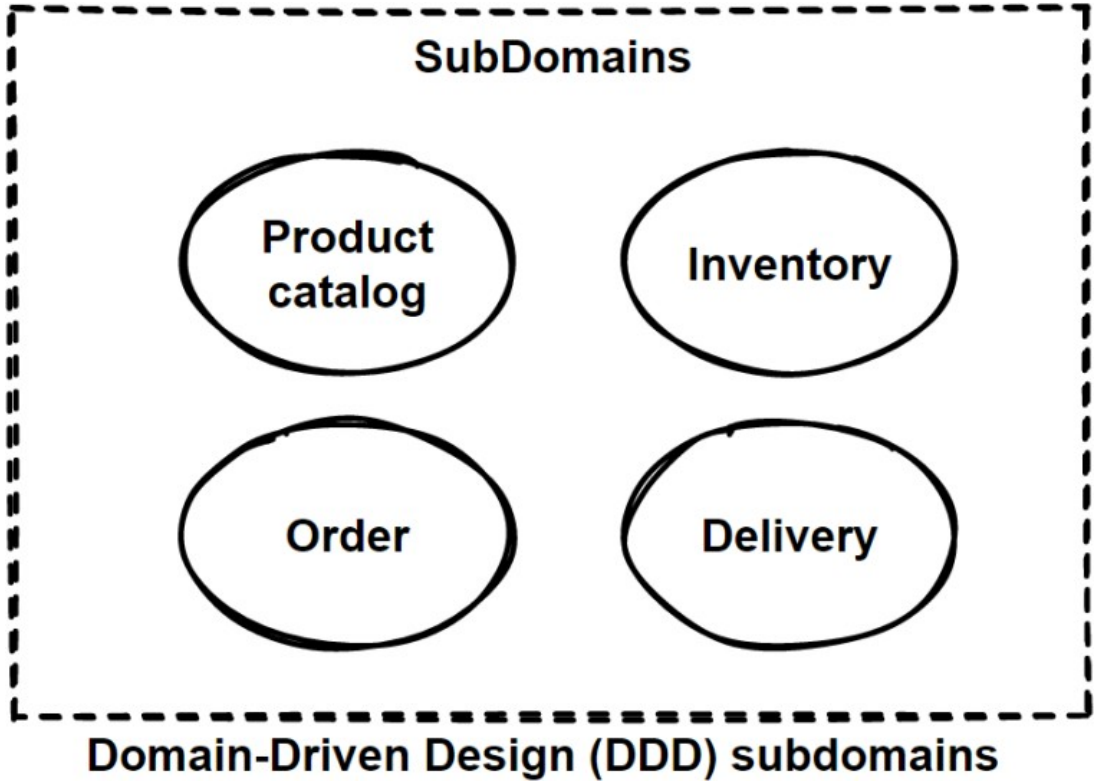
- In Microservices Architecture, **split the application** as a set of **loosely coupled services**.
- **2 Prerequisite** of **decomposition** of microservices:
- **Services** must be **cohesive**. A service should implement a small set of strongly related functions.
- **Services** must be **loosely coupled** - each service as an API that encapsulates its implementation.
- **"Decompose by Business Capability"** pattern offer:
- **Define** services **corresponding** to **business capabilities**.
- A **business capability** is a concept from **business architecture modeling**.
- A **business** service should **generate value**.



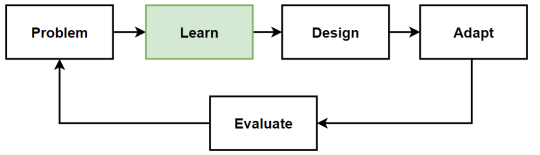
Microservices Decomposition Pattern – Decompose by Subdomain



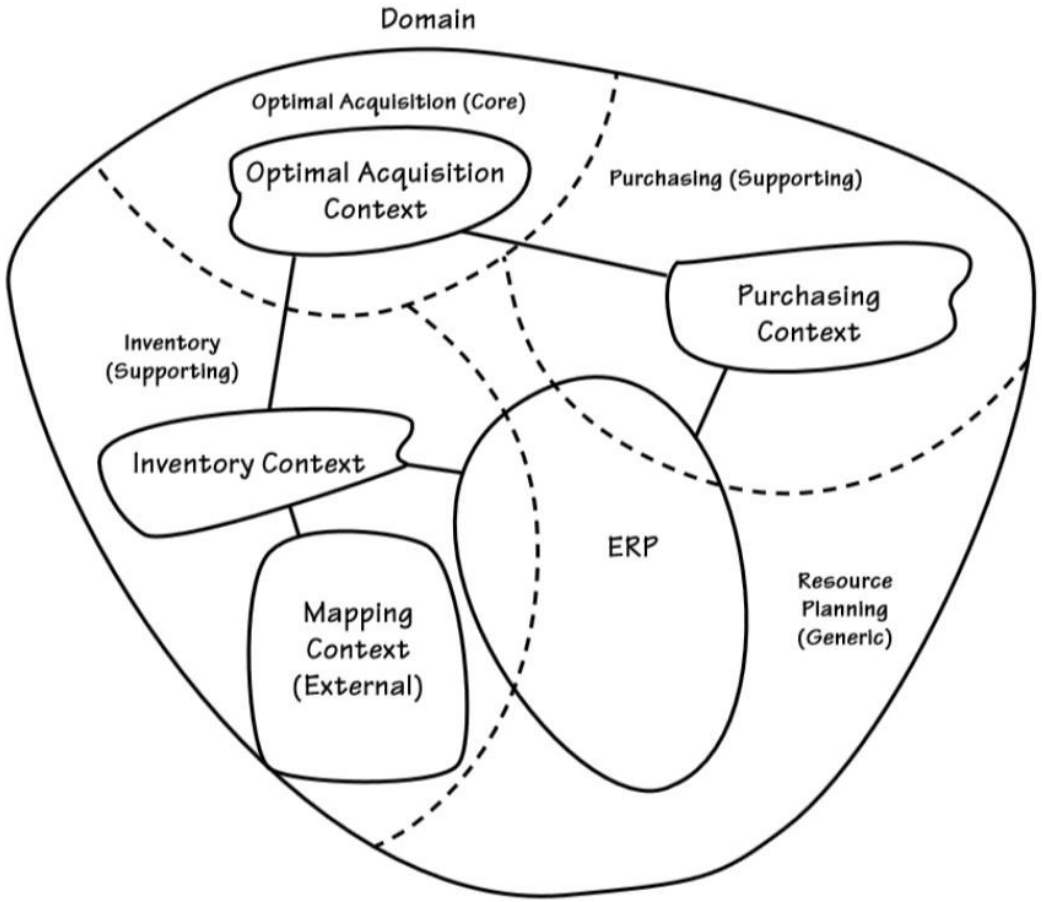
- **Services** must be **cohesive**. A service should implement a small set of strongly related functions.
- **Services** must be **loosely coupled** - each service as an API that encapsulates its implementation.
- "**Decompose by Subdomain**" pattern offer:
- **Define** services **corresponding** to **Domain-Driven Design (DDD) subdomains.**
- DDD refers to the application's problem space, the **business as the domain**. A **domain** is consists of **multiple subdomains**.
- **Each subdomain** corresponds to a **different part** of the **business**.



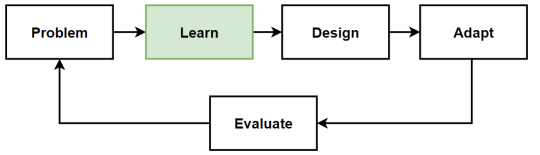
Bounded Context Pattern (Domain-Driven Design)



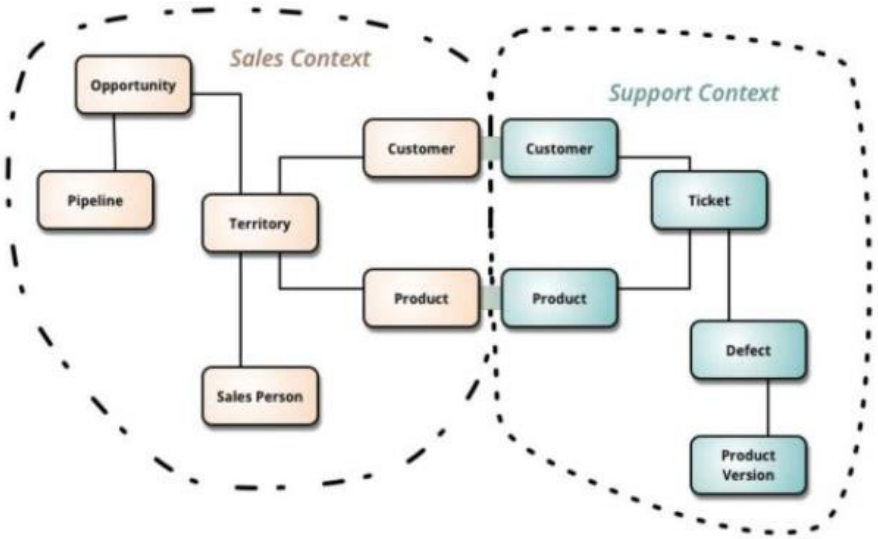
- **DDD - Bounded Context Pattern** which is one of the main pattern that we mainly use when **decomposing microservices**.
- **Domains** are require **high cooperation** and have a certain complexity by nature are called **collaborative domains**.
- DDD has **2 phases**, **Strategic** and **Tactical DDD**.
- **Strategic DDD**, we define the large-scale model of the system, defining to the business rules that allow designing loosely coupling units and the context map between them.
- **Tactical DDD** focuses on implementation and provides design patterns that we can use to build the software implementation.
- Include concepts such as **entity**, **aggregate**, **value object**, **repository**, and **domain service**.



Bounded Context Pattern (Domain-Driven Design)-2



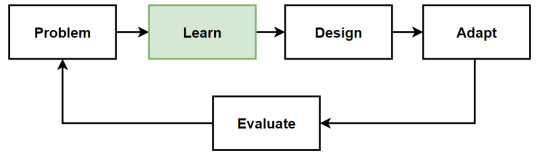
- **DDD** domain defines its **own common language** and divides boundaries into specific, independent components. Common language is called **ubiquitous language**, and independent units are called **bounded context**.
- **DDD** is solving a **complex** problem is to break the problem into **sm** relatively easy.
- **A complex domain** may contain **sub domains**. And some of sub other for **common rules** and **responsibilities**.
- **Bounded Context** is the grouping of closely related scopes tha we
- **Bounded context** is the **logical boundary** that represents the that **are self- consistent** and as **independent** as possible.



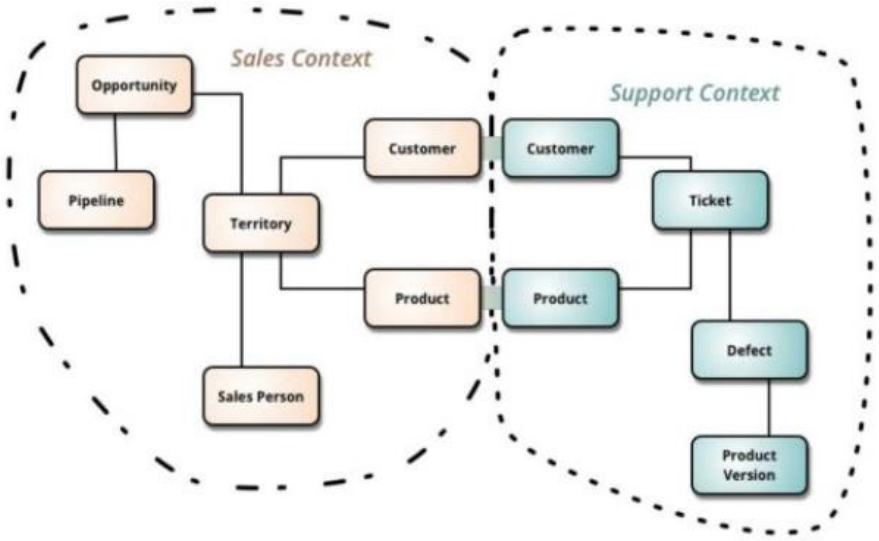
DDD deals with large models by dividing them into different Bounded Contexts and being explicit about their interrelationships.

http://martinfowler.com/bliki/BoundedContext.html

Identify Bounded Context Boundaries for Each Microservices



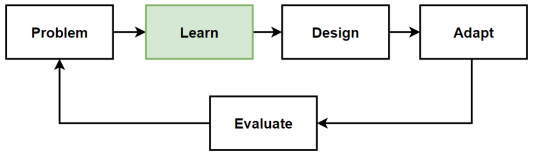
- How we can identify **Bounded Context** ?
- To identify bounded contexts use **DDD**.
- **In DDD**, use the **Context Mapping pattern** for identification of bounded contexts.
- **With Context Mapping Pattern**, we can identify the whole bounded contexts in the application and with their **logical boundaries.**
- The solution is using **Context Mapping.**
- **The Context Map** is a way to define **logical boundaries** between domains.



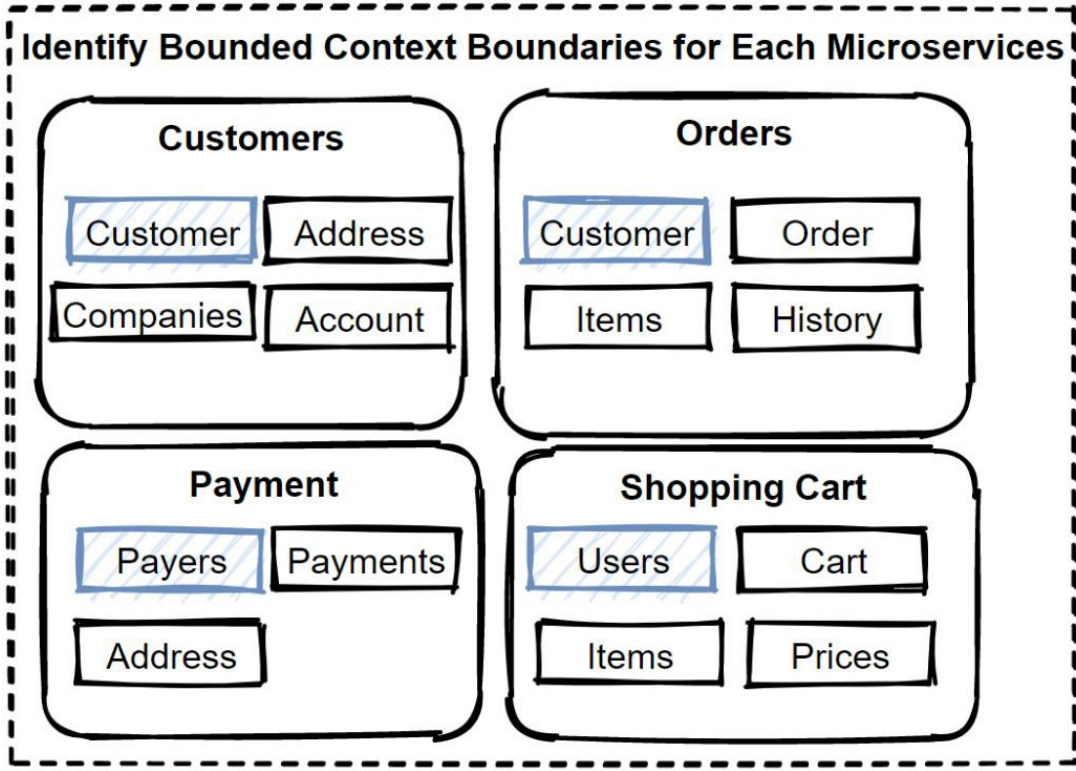
DDD deals with large models by dividing them into different Bounded Contexts and being explicit about their interrelationships.

http://martinfowler.com/bliki/BoundedContext.html

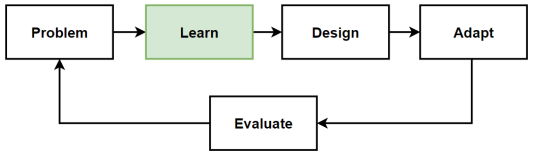
Identify Bounded Context Boundaries for Each Microservices-2



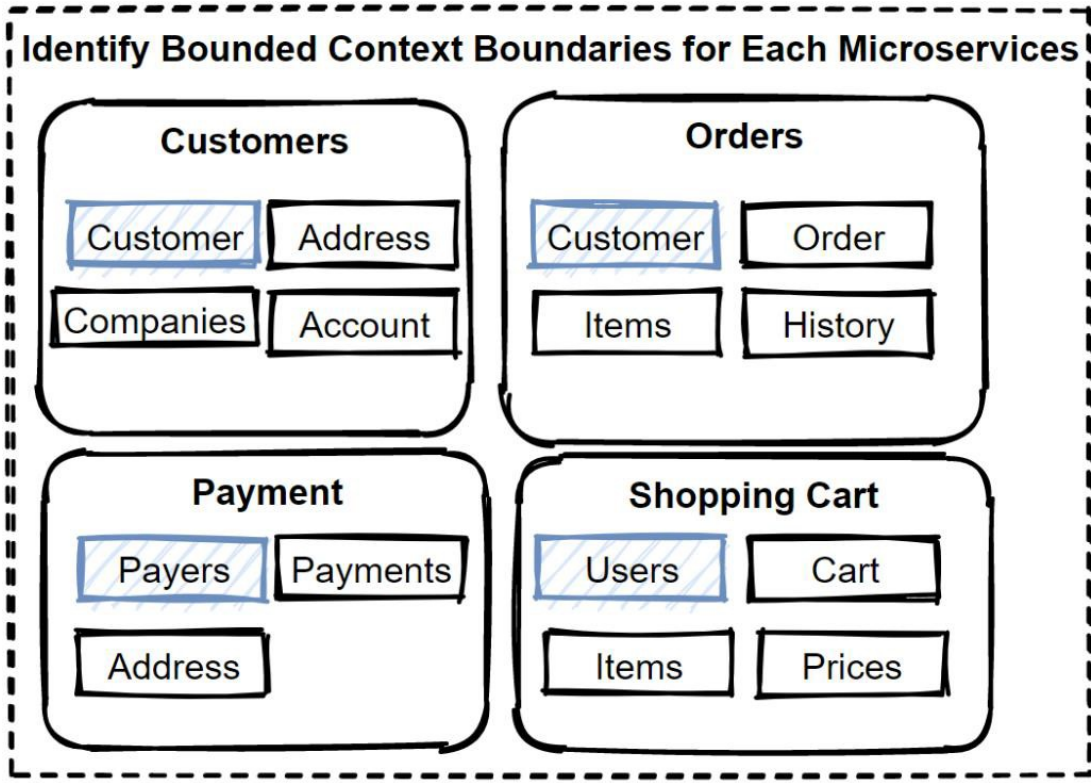
- **Identify** the **Bounded Contexts** by talking to the **domain experts** and using some clues.
- Once defined the Bounded Contexts, iterate design, those are **not immutable**.
- **Reshape** your **Bounded Contexts** by talking to the domain experts and consider **refactoring's** with the changing conditions.
- Its crucial to discuss with **domain experts** to defining **domains** and **sub domains**.
- **Evaluate Bounded Context** with the **domain experts** will help you identify to microservices.
- **Sub domains** inside of the **Bounded Context** are representing same data but naming differently due to domain experts areas.
- Should **discuss several domain experts** for their expertise areas.



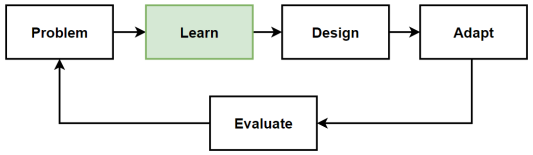
A Bounded Context == A Microservice ?



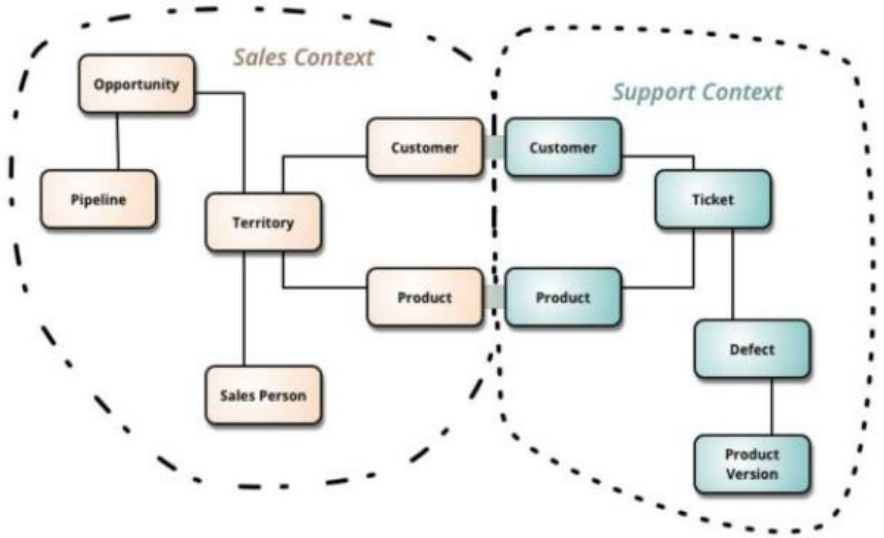
- **No right answer** to this question under all circumstances.
- **Bounded Context** can create **more than one Microservice**.
- Decision to be made based on the **microservice's** need for **scalability** and **independence**.
- Since Bounded Context defines the boundaries of the domain, a **Microservice** determines the **technical** and **organizational boundaries**.
- Similar to Microservices, **Bounded Contexts** are **autonomous** and **responsible** by certain **domain capability**.
- **Context Mapping** and the **Bounded Context pattern** are good approaches for **identifying microservices**.



Using Domain Analysis to Model Microservices



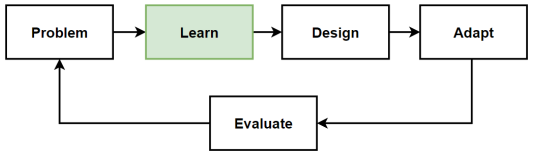
- **Microservices** should be designed by **business capabilities** and should have **loose coupling** and **autonomous services**.
- We **can change** a particular microservices **without affecting** other services. Each service can be change **independently**.
- **Domain-driven design (DDD)** provides a set of methodology that we can follow the principles and create a well-designed microservices.
- Follow **DDD-Bounded Context** which following **Context Mapping Pattern** and **decompose** by **sub domain** models patterns.



DDD deals with large models by dividing them into different Bounded Contexts and being explicit about their interrelationships.

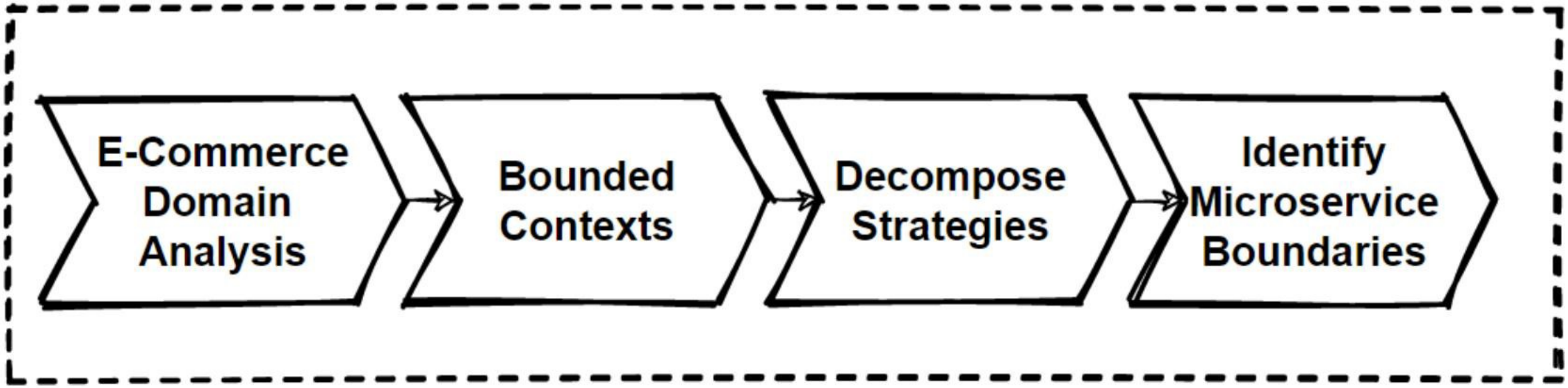
http://martinfowler.com/bliki/BoundedContext.html

Checklist After Decompose Microservices



- **Microservice should do "one thing"**
  No certain process that will produce the right design. But Each service should has a single responsibility. Think deeply about our business domain, bounded contexts and sub domain models patterns.
- **Microservice size should not too big and not too small**
  Start from a carefully designed domain model and group small sub models with obeying this rule. Each service is small enough that it can be built by a small team working independently.
- **Avoid Chatty Communication**
  When you splitting functionality into two services, if those services becomes overly chatty communications, then its good to combine them into 1 service.
- **No Locking Dependencies**
  If your services has inter-service dependencies more than 2 or 3, and if those are required to move and deploy together that means there are pain points of your design and its good to re-think again.
- It should always be possible to deploy a microservice without re-deploying any other services. Services should not be tightly coupled, and can evolve independently.

Microservices Decomposition Path

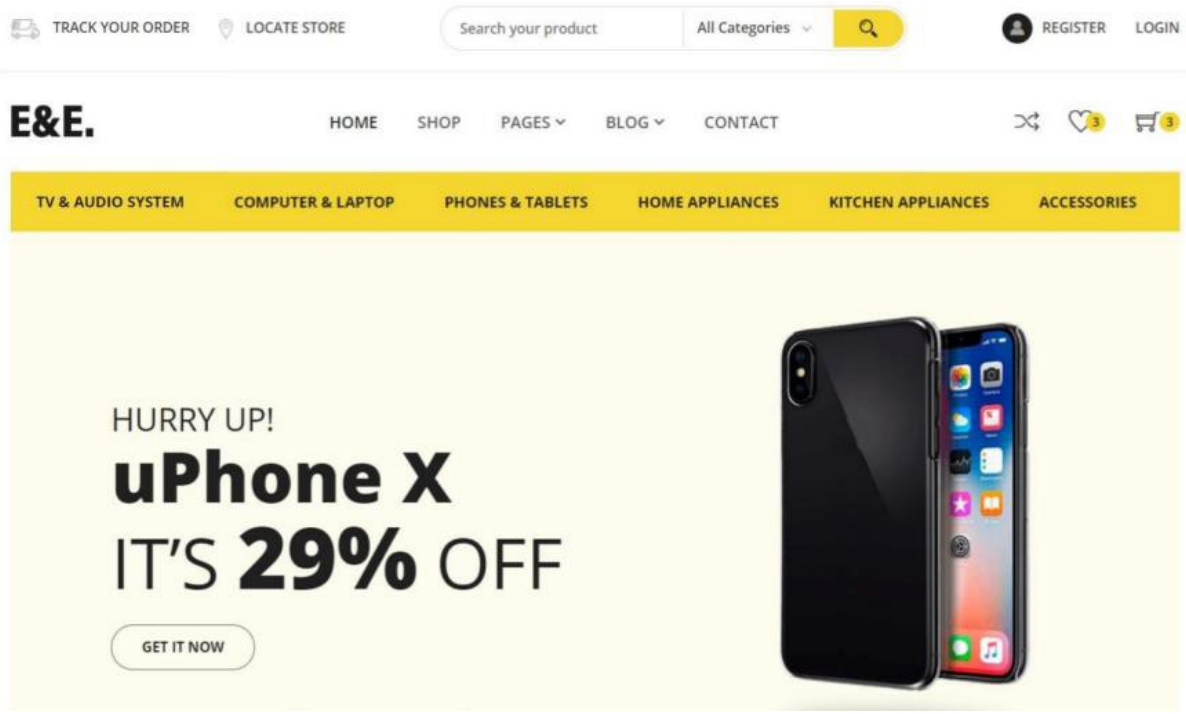


Understand E-Commerce Domain

- Our Domain: E-Commerce
- Understand Domain and Decompose small pieces
  - Use Cases
  - Functional Requirements

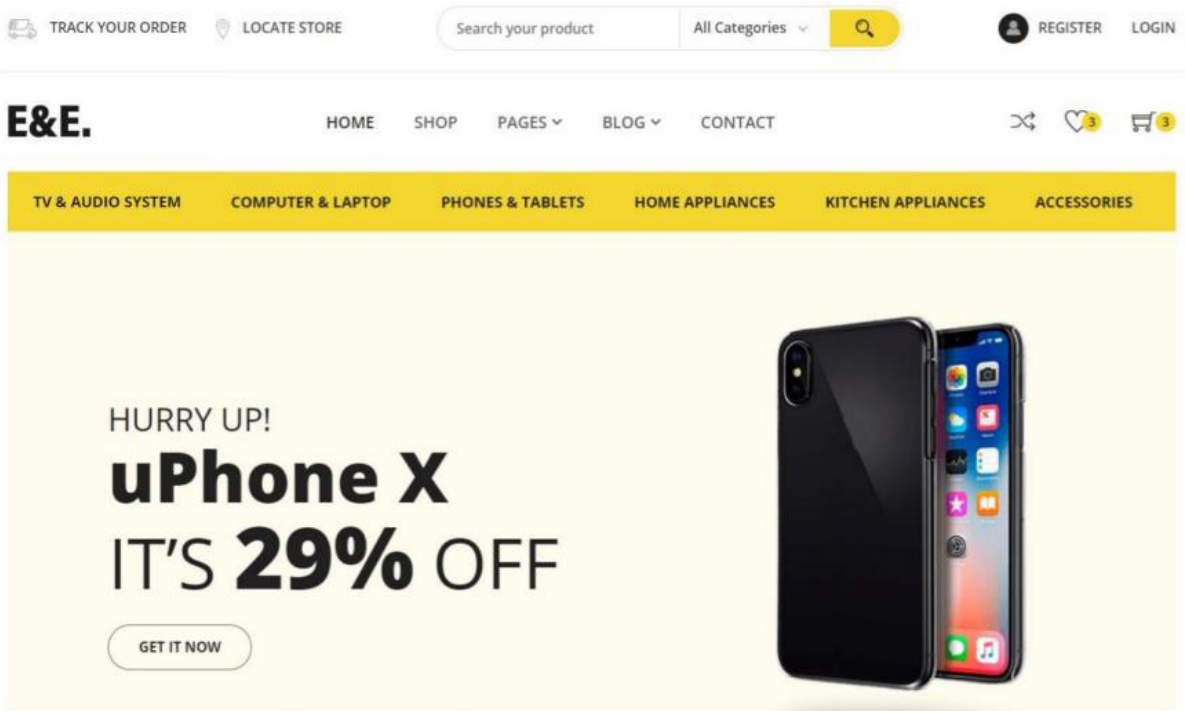
Identify steps;

- Requirements and Modelling
- Identify User Stories
- Identify the Nouns in the user stories
- Identify the Verbs in the user stories



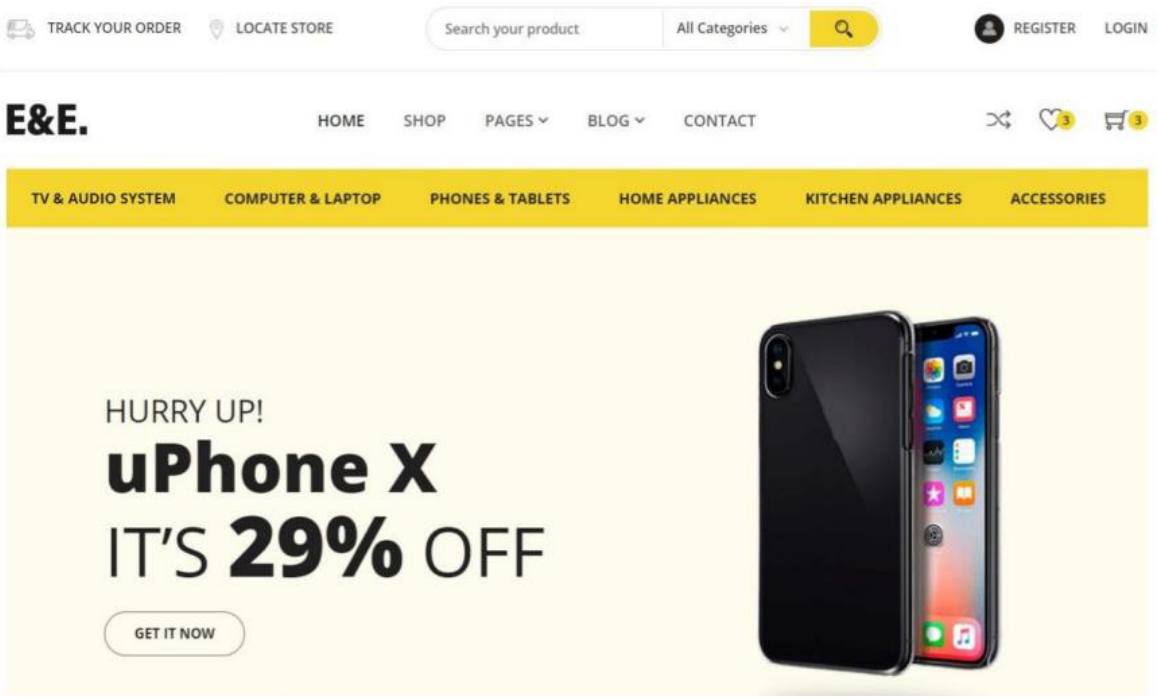
Understand E-Commerce Domain: Functional Requirements

- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts and see the total cost all for all of the items in shopping cart
- Checkout the shopping cart and create an order
- List my old orders and order items history



Understand E-Commerce Domain: User Stories (Use Cases)

- As a user I want to list products
- As a user I want to filter products as per brand and categories
- As a user I want to put products into the shopping cart so that I can check out quickly later
- As a user I want to apply coupon for discounts and see the total cost all for all of the items that are in my cart
- As a user I want to checkout the shopping cart and create an order
- As a user I want to list my old orders and order items history
- As a user I want to login the system as a user and the system should remember my shopping cart items

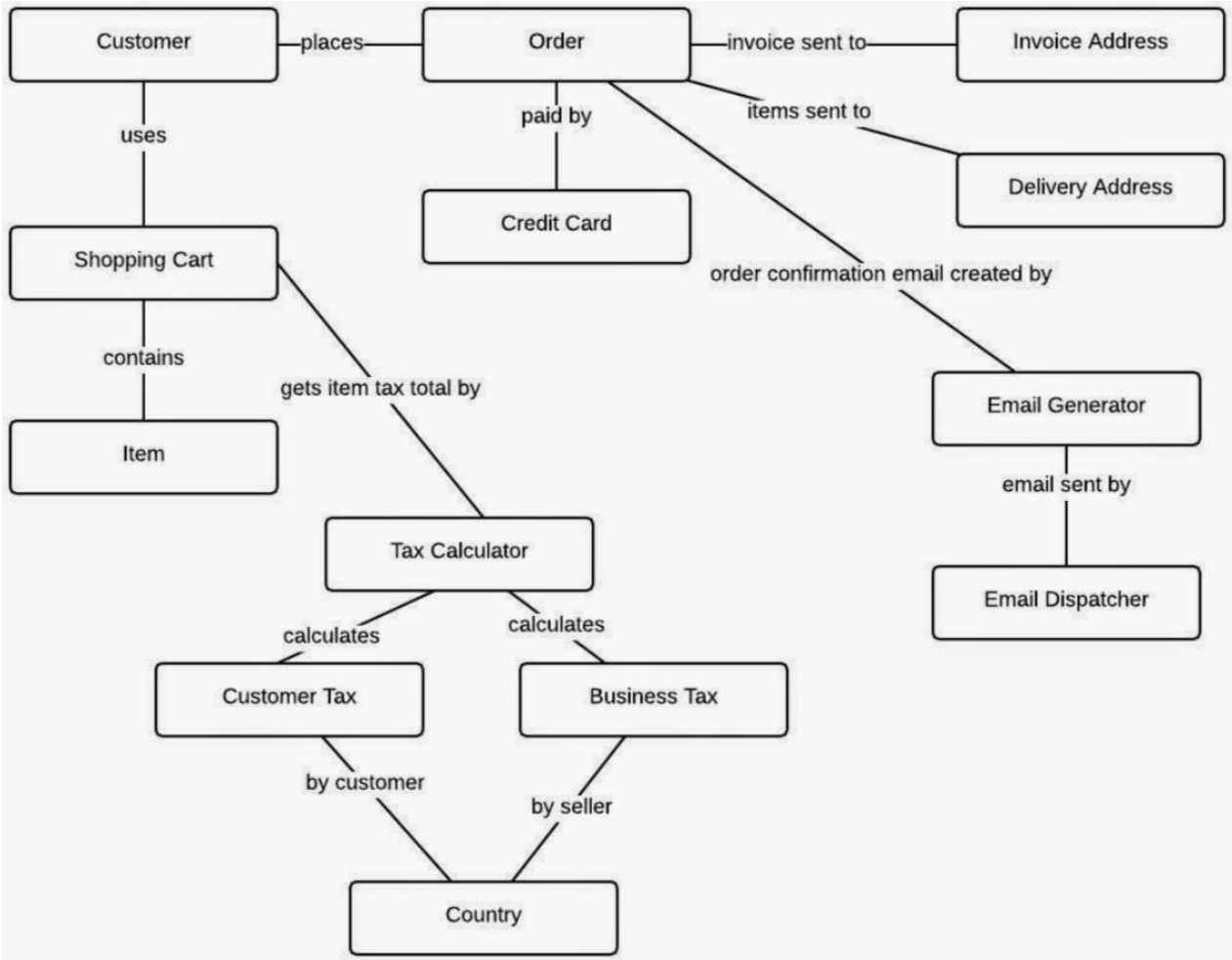


Analysis E-Commerce Domain - Nouns and Verbs

- As a user I want to **list** **products**
- As a user I want to be able to **filter** **products** as per **brand** and **categories**
- As a user I want to **see** the supplier of **product** in the product detail screen with all characteristics of product
- As a user I want to be able to **put** **products** that I want to **purchase** in to the **shopping cart** so I can check out
- As a user I want to **see** the total cost all for all of the **items** that are in my **cart** so that I see if I can afford to buy
- As a user I want to **see** the total cost of each **item** in the **shopping cart** so that I can re-check the price for items
- As a user I want to be able to **specify** the **address** of where all of the products are going to be sent to
- As a user I want to be able to **add** a note to the **delivery address** so that I can provide special instructions
- As a user I want to be able to **specify** my **credit card** information during **check out** so I can pay for the items
- As a user I want system to **tell** me how many items are in **stock** so that I know how many items I can purchase
- As a user I want to **receive** **order** confirmation email with **order** number so that I have proof of purchase
- As a user I want to **list** my old **orders** and **order items history**
- As a user I want to **login** the system as a **user** and the system should remember my shopping cart items

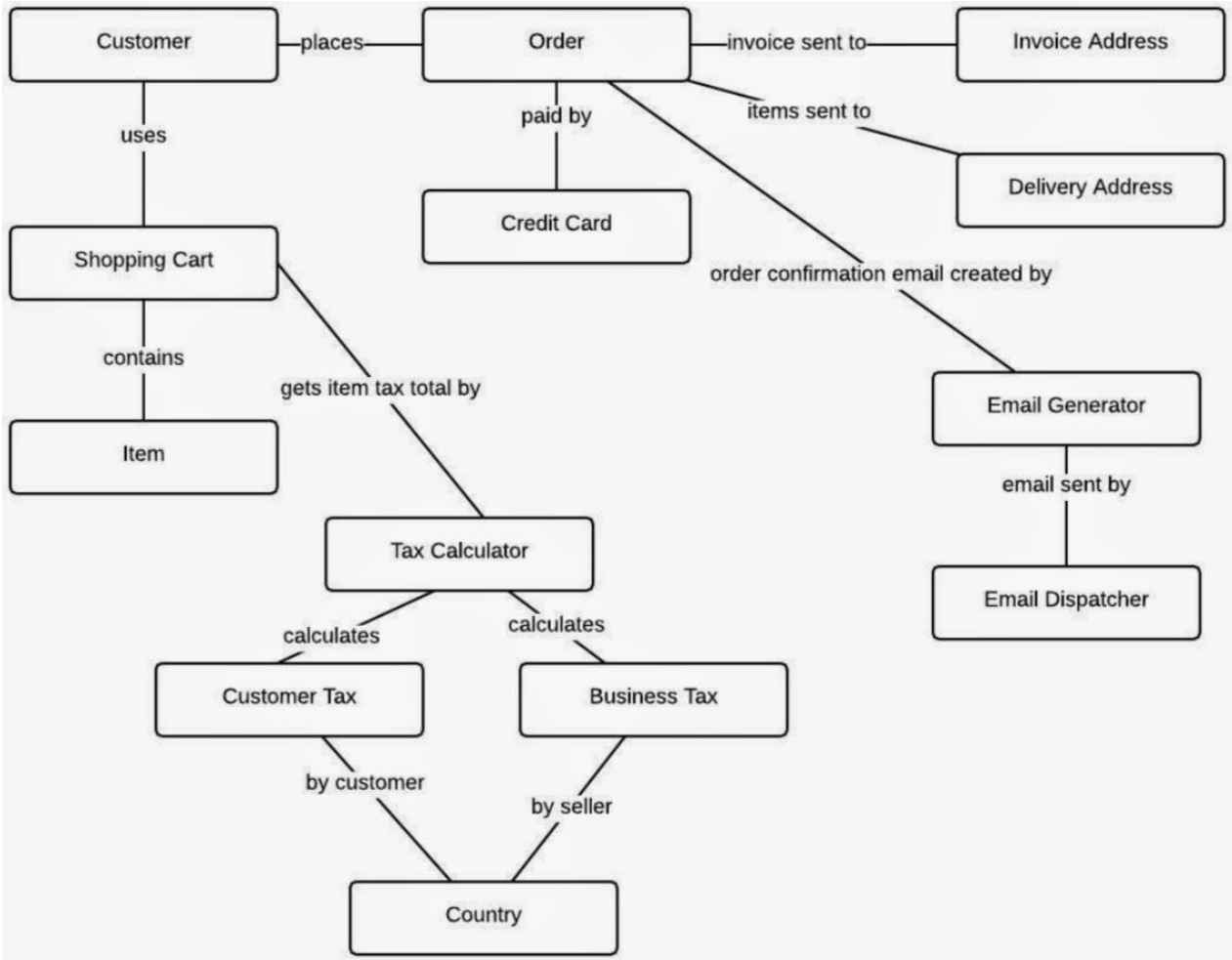
Analysis E-Commerce Domain - Nouns

- Customer
- Order
- Order Details
- Product
- Shopping Cart
- Shopping Cart Items
- Supplier
- User
- Address
- Brand
- Category

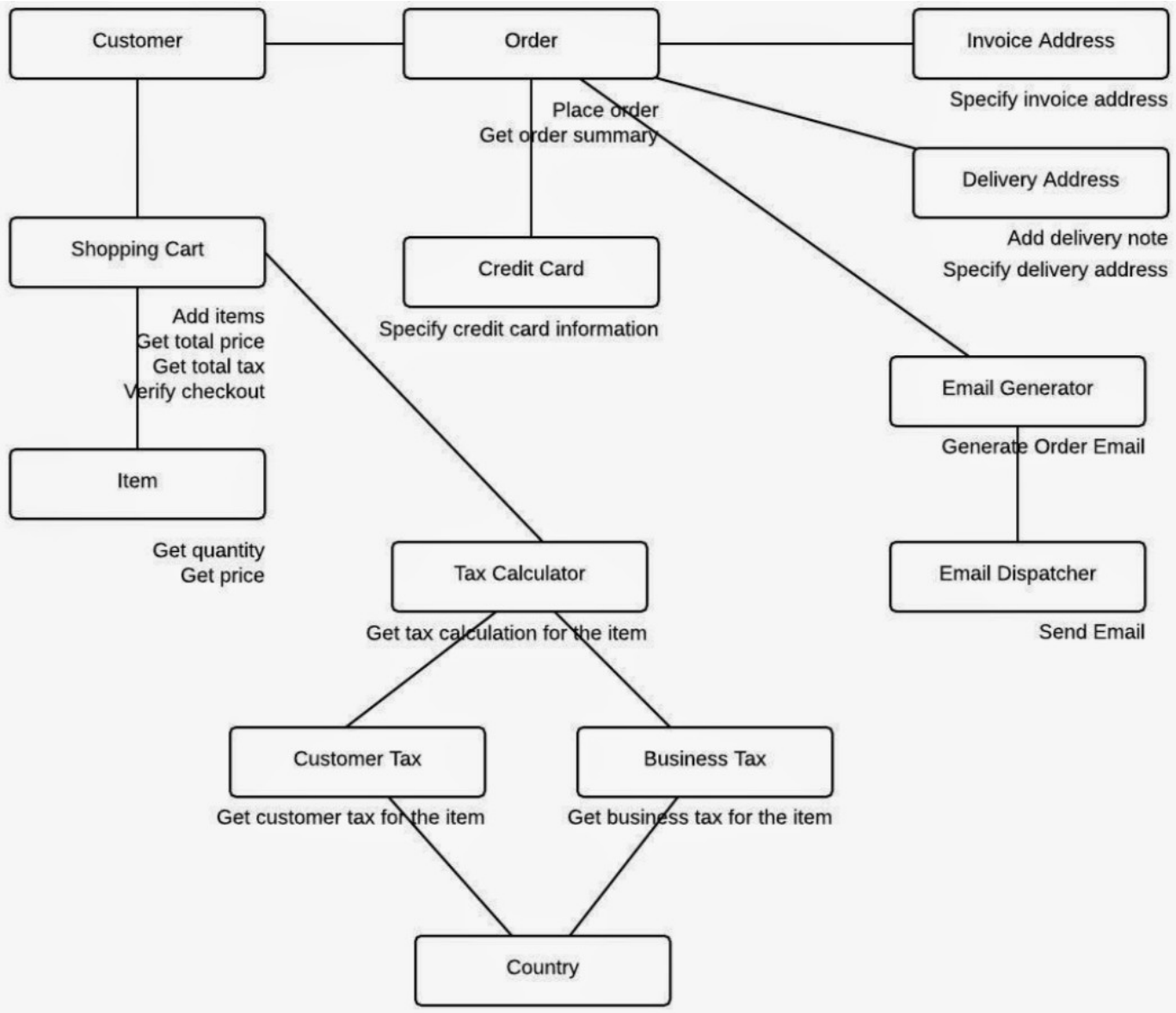


Analysis E-Commerce Domain - Verbs

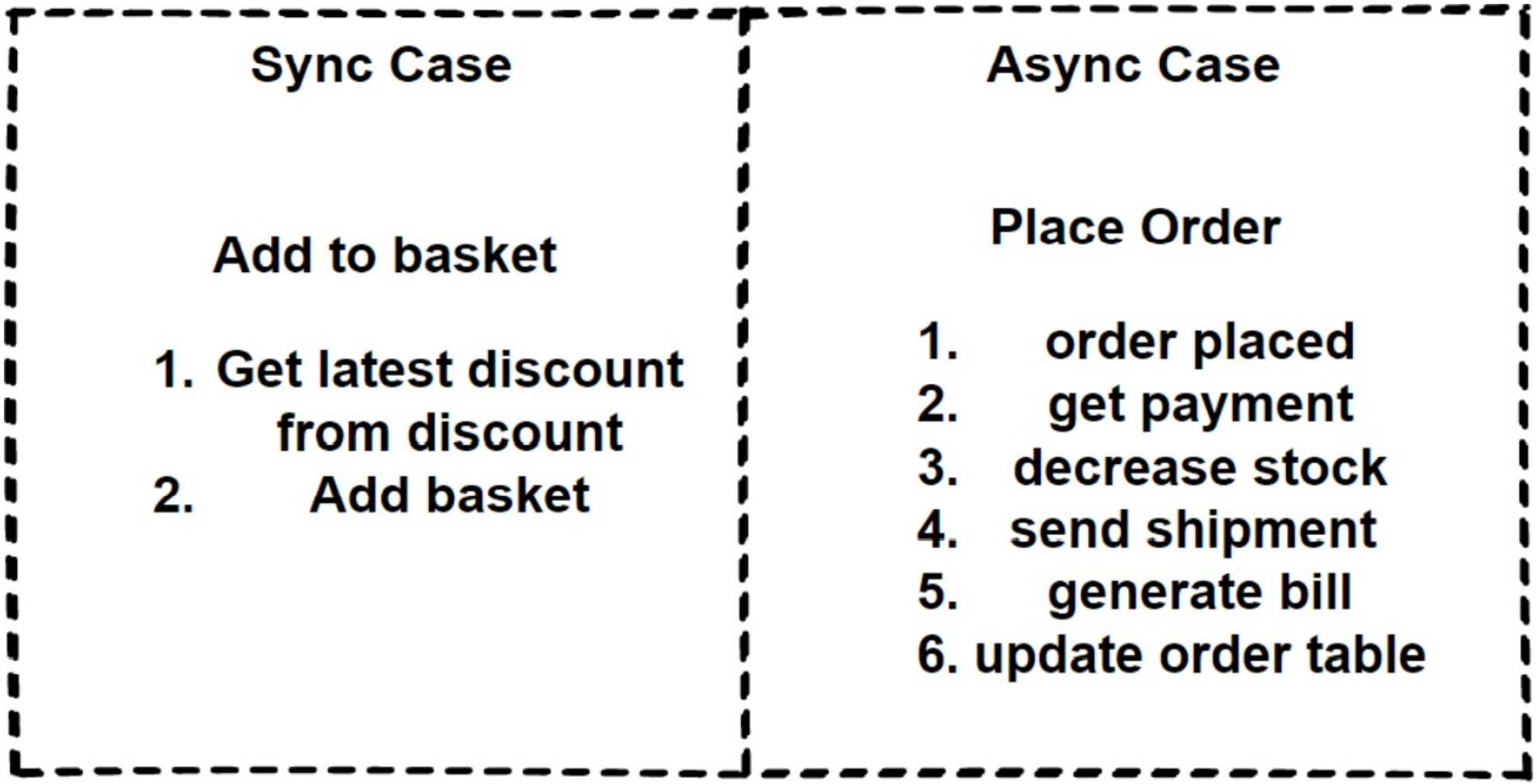
- List products applying to paging
- Filter products by brand, category and supplier
- See product all information in the details screen
- Put products in to the shopping cart
- See total cost for all of the items
- See total cost for each item
- Checkout order with purchase steps
- Specify delivery address
- Specify delivery note for delivery address
- Specify credit card information
- Pay for the items
- Tell me how many items are in stock
- Receive order confirmation email
- List the order and details history
- Login the system and remember the shopping cart items

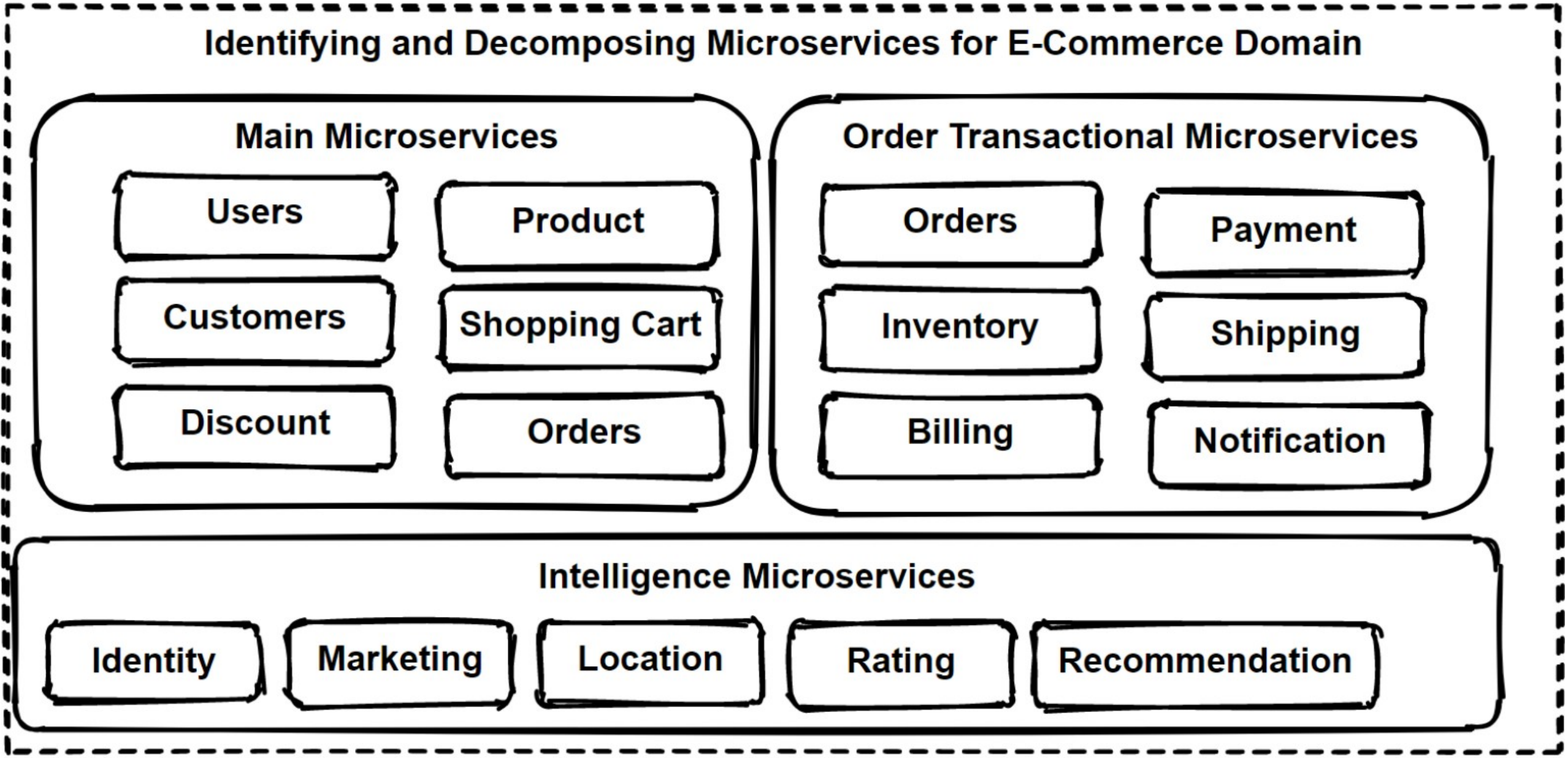


Object Responsibility Diagram



2 Main Use Case of Our E-Commerce Application




Identifying and Decomposing Microservices for E-Commerce Domain
Main Microservices
Users
Product
Customers
Shopping Cart
Discount
Orders
Order Transactional Microservices
Orders
Payment
Inventory
Shipping
Billing
Notification
Intelligence Microservices
Identity
Marketing
Location
Rating
Recommendation

Before Design – What we have in our design toolbox ?

Architectures

- **Microservices Architecture**

Patterns&Principles

- The Database-per-Service Pattern
- Polygot Persistence
- **Decompose services by scalability**
- **The Scale Cube**
- **Microservices Decomposition Pattern**
  - **Decompose by Business Capability**
  - **Decompose by Subdomain**
  - **Bounded**
  - **Context Pattern**
  - **Context Mapping pattern**

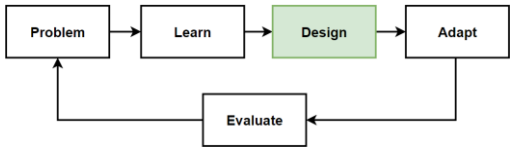
Non-FR

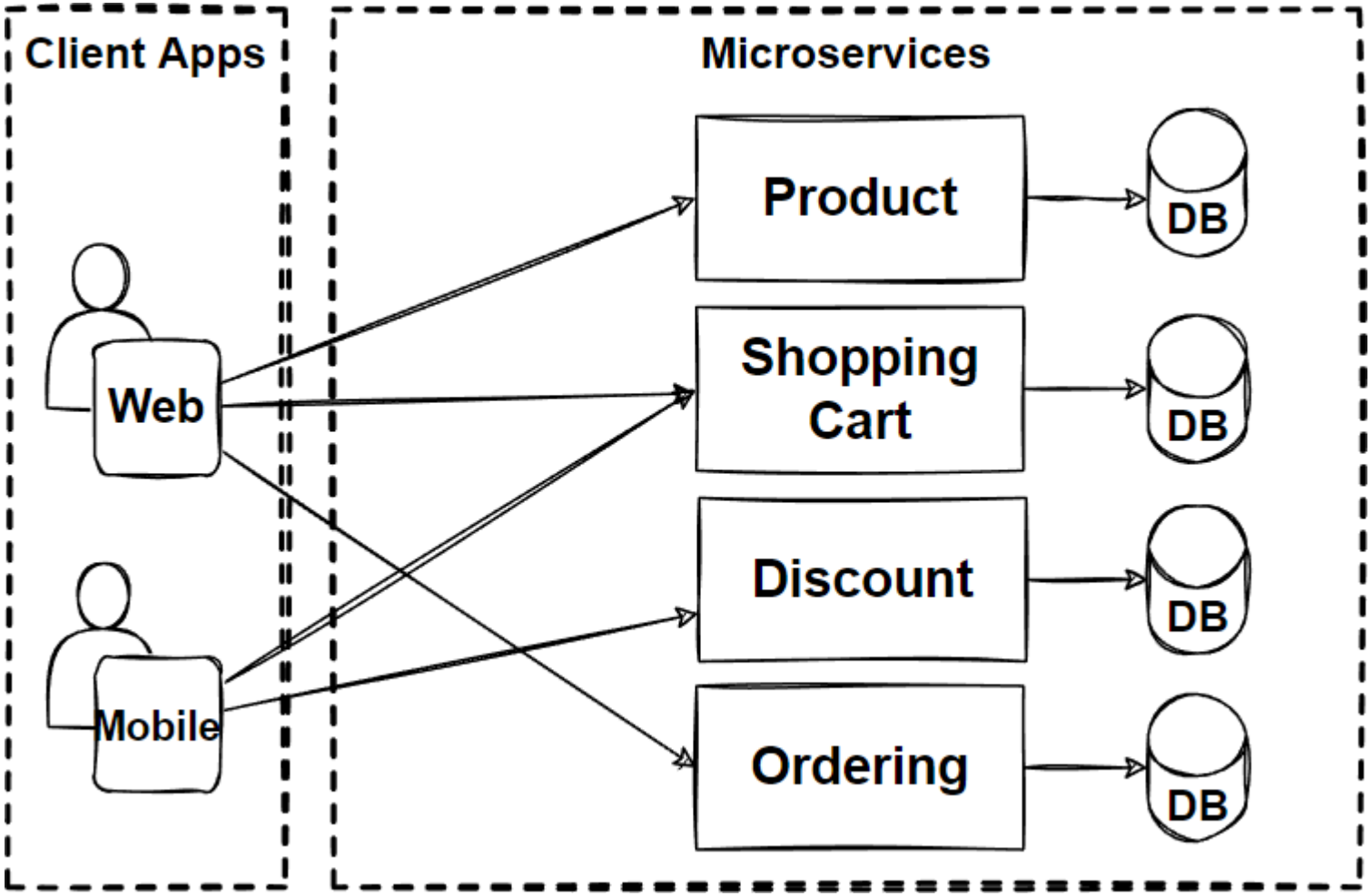
- High Scalability
- High Availability
- Millions of Concurrent User
- Independent Deployable
- Technology agnostic
- Data isolation
- Resilience and Fault isolation

FR

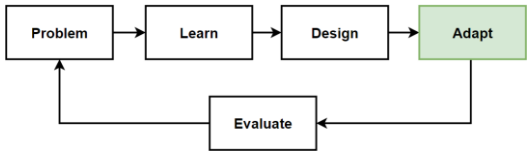
- List products
- Filter products as per brand and categories
- Put products into the shopping cart
- Apply coupon for discounts
- Checkout the shopping cart and create an order
- List my old orders and order items history

Design: Microservices Architecture





Adapt: Microservices Architecture



Frontend SPAs

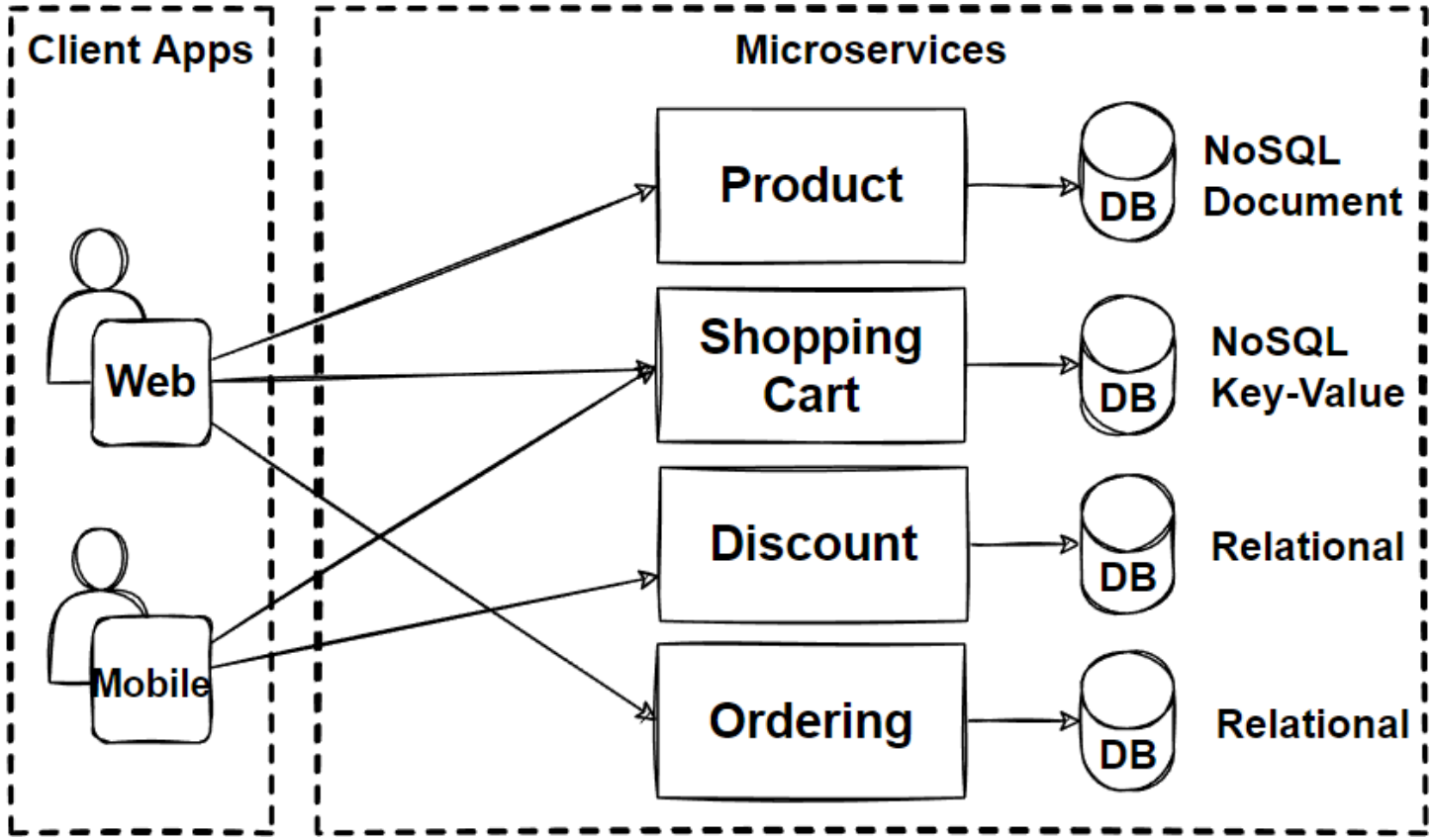
- Angular
- Vue
- React

Backend Microservices

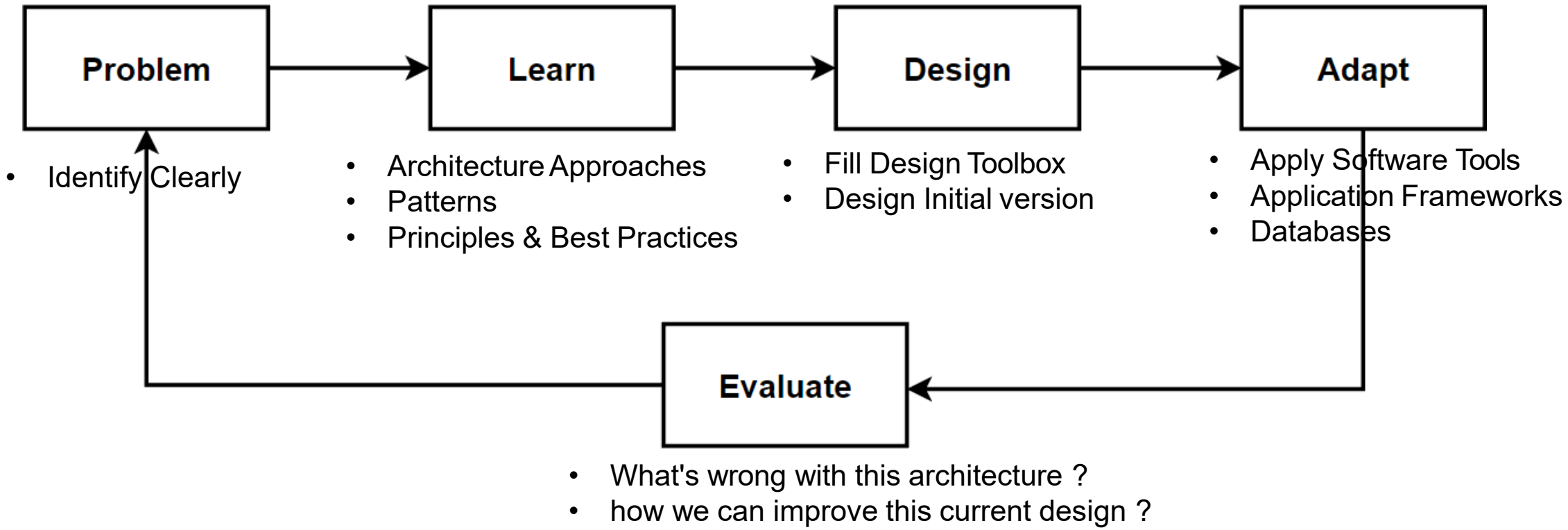
- Java – Spring Boot
- .Net – Asp.net
- JS – NodeJS
- Python – Django, Flask

Database

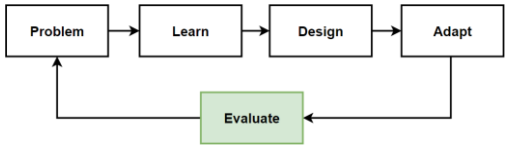
- MongoDB – NoSQL Document DB
- Redis – NoSQL Key-Value Pair
- Postgres – Relational
- SQL Server – Relational



Way of Learning – The Course Flow



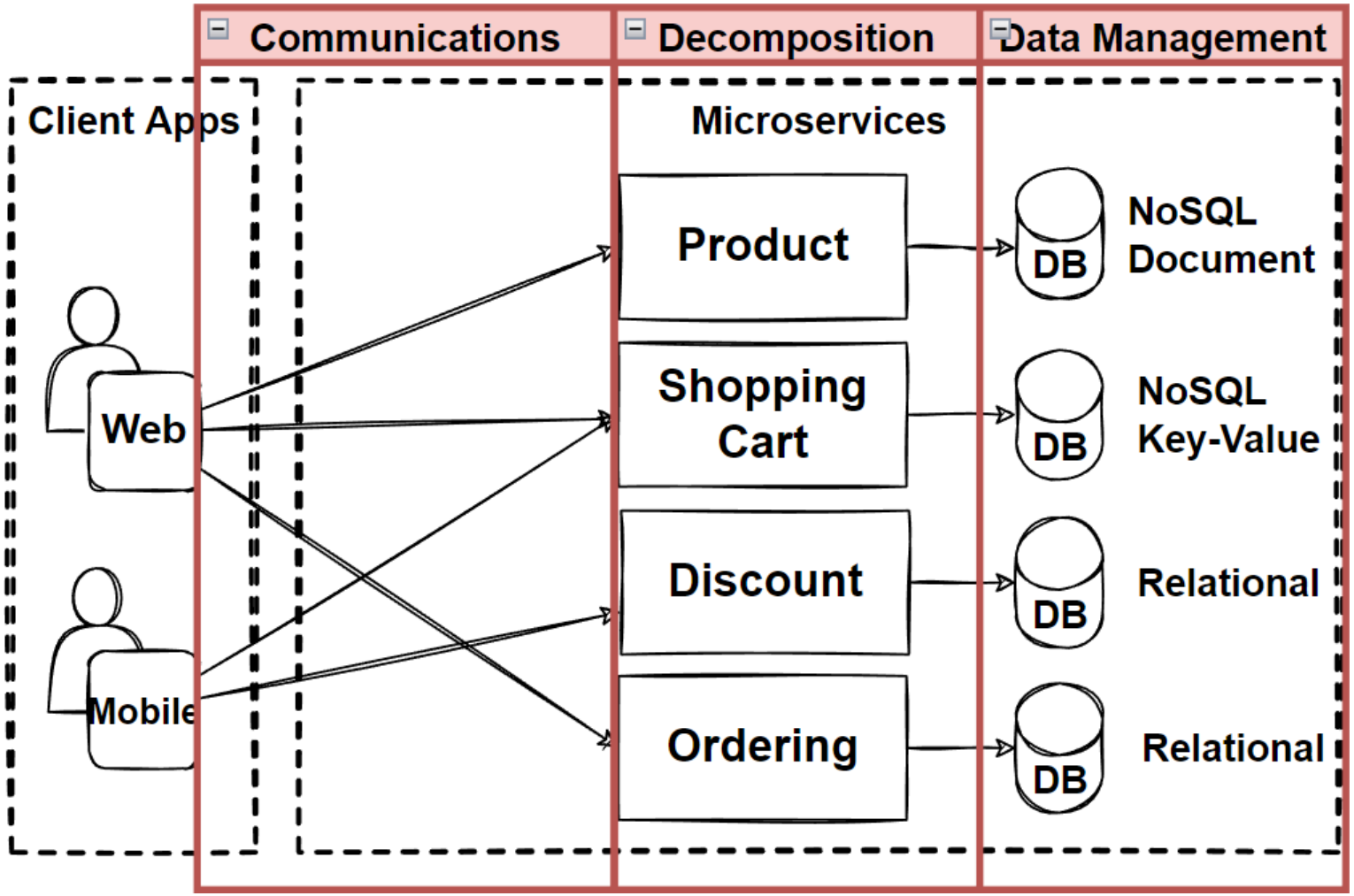
Evaluate: Microservices Architecture



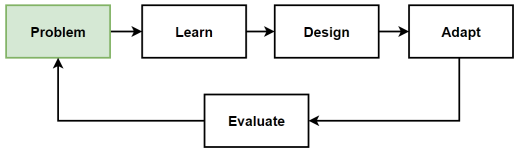
Main Considerations

- Decomposition – Breaking Down Services
- Communications
- Data Management

- Transaction Management
- Deployments
- Resilience



Problem: Direct Client-to-Service Communication



Problems

- Direct Client-to-Service Communication
- Cause to chatty calls from client to service
- Hard to manage invocations from client app.

Solutions

- Well-defined API Design
- Microservices Communication Patterns